

BRISAS DE CALAMUCHITA

Sistema web de gestión de reservas

Informe Final · Proyecto Integrador PPIV

Etapa 4 · Desarrollo, despliegue y documentación final

AUTOR

Jorge Kalas

DOCENTES

Emir García Ontiveros

Kevin Del Bello

INSTITUCIÓN

Instituto de Formación Técnica Superior N° 29

Tecnicatura en Desarrollo de Software

Primer semestre 2026

Índice de contenidos

Resumen ejecutivo	8
Bloque A — Documento base del sistema	10
Descripción del problema	10
Propósito del software	11
Alcance del sistema.....	11
Objetivos del proyecto	12
Objetivo general.....	12
Objetivos específicos	13
Usuarios del sistema	14
Visitante	14
Cliente	14
Administrador.....	14
Funcionalidades del sistema	15
Gestión de usuarios	15
Gestión de reservas (cliente)	15
Gestión administrativa.....	15
Sistema de notificaciones automáticas	16
Análisis de soluciones similares (benchmark).....	16
Plataformas comerciales (Booking, Airbnb).....	16
Gestores profesionales (Lodgify, Smoobu)	17
Sistemas a medida	17
Posicionamiento del proyecto	17
Viabilidad del proyecto	17
Viabilidad técnica	17

Viabilidad económica	18
Viabilidad operativa.....	18
Bloque B — Modelado del sistema.....	19
Diagrama de casos de uso	19
Diagrama de entidad-relación (DER).....	20
Diagrama de clases.....	22
Diagrama de estados	23
Diagrama de secuencia.....	25
Bloque C — Desarrollo e implementación	28
Arquitectura del sistema	28
Las tres capas	28
Comunicación entre capas.....	28
Stack tecnológico — justificación.....	29
React + Vite (frontend).....	29
Node.js + Express (backend)	30
MySQL (base de datos)	31
Tailwind CSS (sistema de estilos).....	31
JWT (autenticación)	32
bcrypt (hash de contraseñas).....	33
Zod (validación de entrada)	33
Docker (entorno de desarrollo).....	34
Decisiones de diseño clave	34
Modelo híbrido cliente/admin con confirmación manual	34
Bloqueo temporal de 2 horas	35
Máquina de estados explícita.....	35
Patrón outbox para notificaciones	36
Cron interno vs cron externo	36

Estructura del proyecto.....	37
Árbol de carpetas.....	37
Modelo de datos detallado.....	38
Tabla usuario	38
Tablas cliente y administrador (extensión)	38
Tabla reserva.....	39
Tablas vehiculo, pago y notificacion.....	39
Diagrama resumen de relaciones	40
Componentes principales del backend	40
Bootstrap del servidor (src/servidor.js)	40
Configuración (src/config/)	40
Rutas (src/rutas/)	40
Controladores (src/controladores/).....	41
Servicios (src/servicios/)	41
Middlewares (src/middlewares/).....	42
Tareas programadas (src/tareas/).....	42
Transacciones críticas	42
Gestión de errores y validaciones.....	43
Validaciones en cliente (frontend)	43
Validaciones de esquema en servidor (Zod)	43
Validaciones de reglas de negocio en servicios	44
Constraints en base de datos.....	44
Confirmación antes de operaciones destructivas	45
Mensajes claros e informativos	45
Componentes principales del frontend	45
Routing (src/App.jsx).....	45
Componentes reutilizables (src/componentes/).....	46

Contexto global (src/contexto/ContextoAuth.jsx)	46
Cliente HTTP (src/api/cliente.js)	46
Módulos de API por dominio	47
Hook useApi (src/hooks/useApi.js)	47
Demostración del CRUD funcional	47
Operaciones sobre la entidad Usuario	48
Operaciones sobre la entidad Reserva	48
Operaciones sobre Vehículo, Pago y Notificación	49
Evidencia de funcionamiento del CRUD	49
Pruebas unitarias	50
Tres pruebas unitarias representativas	50
Resumen de cobertura	52
Pruebas unitarias del frontend (Vitest)	52
Pruebas de integración	53
Dos pruebas de integración representativas	53
Resumen de las 40 pruebas E2E	55
Ejecución del pipeline completo	55
CI/CD y despliegue	56
Pipeline de CI con GitHub Actions	56
Despliegue en producción	56
Separación de entornos	57
Bloque D — Manuales del sistema	58
Manual de usuario — Visitante y Cliente	58
Introducción	58
Requisitos mínimos	58
Pantalla principal	58
Flujo de uso 1: Consultar disponibilidad como visitante	61

Flujo de uso 2: Crear cuenta	61
Flujo de uso 3: Solicitar una reserva (cliente).....	62
Flujo de uso 4: Ver "Mis reservas"	64
Flujo de uso 5: Cancelar una reserva.....	65
Manual de usuario — Administrador.....	65
Estructura del panel administrativo	66
Flujo de uso 1: Confirmar una reserva	67
Flujo de uso 2: Cancelar una reserva (admin).....	68
Flujo de uso 3: Registrar check-in.....	68
Flujo de uso 4: Registrar check-out.....	69
Flujo de uso 5: Filtrar el listado	69
Manual de usuario — Mensajes y alertas frecuentes.....	69
Mensajes informativos (azules / neutros)	69
Mensajes de error / advertencia (rojos).....	70
Mensajes de confirmación (modales).....	70
Manual técnico — Instalación y puesta en marcha local.....	71
Requisitos previos.....	71
Paso 1: Clonar el repositorio	71
Paso 2: Levantar la base de datos MySQL local.....	71
Paso 3: Configurar las variables de entorno del backend.....	72
Paso 4: Instalar dependencias del backend	72
Paso 5: Ejecutar migraciones y datos iniciales.....	72
Paso 6: Iniciar el backend	73
Paso 7: Configurar e iniciar el frontend	73
Credenciales de prueba (entorno local)	73
Comandos útiles del proyecto	74
Resolución de problemas frecuentes (local).....	74

Manual técnico — Despliegue productivo	75
Fase 1: TiDB Cloud (base de datos MySQL)	75
Fase 2: Render (backend Node.js).....	75
Fase 3: Vercel (frontend estático)	76
Fase 4: Conectar Vercel con Render (CRUCIAL)	76
Fase 5: Validación end-to-end	77
Conclusiones	78
Bibliografía	80
Libros y artículos.....	80
Documentación oficial consultada	81
Anexos.....	82
Anexo A — Acceso al sistema en producción.....	82
Anexo B — Credenciales de prueba para el coloquio.....	82
Administrador.....	82
Clientes con reservas activas.....	83
Secuencia recomendada para demostrar el sistema en vivo.....	83
Anexo C — Enlaces a los diagramas UML en draw.io	83
Anexo D — Estructura del repositorio.....	84
Anexo E — Resumen rápido de comandos	85
Anexo F — Notas finales para el equipo evaluador	85
Anexo G — Uso de herramientas de inteligencia artificial.....	86
Anexo H — Documentos de trabajo de etapas anteriores	88
Agradecimientos.....	89

Resumen ejecutivo

El presente informe documenta de forma íntegra el desarrollo del sistema web "Brisas de Calamuchita", una solución informática diseñada a medida para gestionar las reservas de una propiedad de alquiler turístico ubicada en Santa Rosa de Calamuchita, provincia de Córdoba. El trabajo abarca el ciclo de vida completo del proyecto: relevamiento del problema, modelado conceptual, decisiones de arquitectura, implementación, pruebas automatizadas, integración continua y despliegue en un entorno productivo accesible públicamente.

El problema inicial puede sintetizarse en una constatación operativa: la gestión de reservas se realizaba de manera manual, mediante WhatsApp, llamadas telefónicas y anotaciones en papel. Esta operatoria generaba reservas solapadas, cancelaciones no registradas, ausencia de trazabilidad financiera y una experiencia inconsistente para el cliente. La hipótesis que articula este proyecto sostiene que un sistema digital diseñado específicamente para el caso de uso —y no una plataforma genérica como Booking o Airbnb— puede resolver simultáneamente la organización interna del negocio y la captación directa de turistas sin pagar comisiones de intermediación.

La solución implementada es una aplicación web de tres capas construida con React en el frontend, Node.js con Express en el backend y MySQL como motor de base de datos. La arquitectura sigue un patrón de separación de responsabilidades: el cliente consume una API REST stateless protegida con JSON Web Tokens, y la persistencia se centraliza en una base de datos relacional con ocho tablas y un conjunto explícito de invariantes garantizadas por restricciones de integridad referencial.

El sistema introduce un diferencial competitivo respecto de las plataformas comerciales: un modelo híbrido de confirmación, en el cual la reserva atraviesa una máquina de estados con seis nodos (Pendiente, Confirmada, En curso, Finalizada, Cancelada, No Show) y el administrador conserva control manual sobre la transición crítica de Pendiente a Confirmada. Este diseño permite a los propietarios validar pagos, identidad y disponibilidad real antes de comprometer la propiedad, sin sacrificar la inmediatez de la solicitud online.

La calidad del software se garantiza mediante una suite de 152 pruebas automatizadas, distribuidas en 64 pruebas unitarias de backend (Jest), 40 pruebas de integración end-to-end (Jest + Supertest) y 48 pruebas de componentes frontend (Vitest + React Testing Library), ejecutadas en cada push al repositorio a través de un pipeline de GitHub Actions. El sistema se

encuentra desplegado en producción en una arquitectura serverless distribuida: el frontend en Vercel, el backend en Render y la base de datos en TiDB Cloud, con notificaciones por correo electrónico vía SMTP real (Gmail).

Bloque A — Documento base del sistema

Descripción del problema

La propiedad "Brisas de Calamuchita" es una casa de alquiler turístico ubicada en Malvinas Argentinas 189, Santa Rosa de Calamuchita, provincia de Córdoba. Posee capacidad para entre cuatro y diez huéspedes, una cochera con capacidad para un solo vehículo y un patio amplio. Se alquila por temporadas, principalmente durante los meses de verano (diciembre a marzo), aunque también recibe huéspedes los fines de semana largos del resto del año.

Actualmente, su administrador gestiona la totalidad de las reservas de manera manual. El flujo operativo es el siguiente: un potencial huésped contacta por WhatsApp, Instagram o por recomendación directa; consulta disponibilidad en un cuaderno o agenda; el administrador le responde indicando si hay lugar y le pide datos personales y monto de seña; el huésped transfiere la seña; el administrador anota la reserva en el cuaderno; y, eventualmente, se vuelven a comunicar para confirmar la llegada.

Este esquema, aunque funciona, presenta cuatro patologías recurrentes que motivaron el proyecto. En primer lugar, las reservas solapadas: cuando dos potenciales huéspedes consultan por las mismas fechas con minutos de diferencia, es habitual que el propietario les responda afirmativamente a ambos antes de bloquear formalmente las fechas. En segundo lugar, las cancelaciones no registradas: cuando un huésped desiste por mensaje pero el propietario olvida tachar la reserva del cuaderno, la propiedad aparece como ocupada en sus registros pero está realmente disponible. En tercer lugar, la pérdida de información: si el propietario quiere saber cuántos huéspedes recibió el año pasado, o cuántos llegaron con vehículo, o quiénes son sus clientes recurrentes, no tiene cómo extraer esa información de un cuaderno manuscrito. Y en cuarto lugar, la ausencia de visibilidad pública: el alojamiento depende del boca a boca y de Instagram, sin un sitio propio donde los turistas puedan informarse y consultar disponibilidad sin necesidad de iniciar una conversación.

A este conjunto de problemas operativos se suma una restricción económica: las plataformas comerciales como Booking o Airbnb retienen comisiones de entre el quince y el veinte por ciento por reserva, lo que en el contexto de una propiedad familiar representa un impacto financiero relevante. La hipótesis del proyecto es que una solución digital propia puede eliminar simultáneamente los problemas operativos y la dependencia comercial.

Propósito del software

El propósito del sistema desarrollado es centralizar y optimizar la gestión de reservas de la propiedad, facilitando tanto la interacción con los clientes como la administración interna, mediante una solución digital accesible, organizada y confiable. La elección del término "optimizar" antes que "automatizar" no es accidental: refleja una decisión de diseño que se explica en detalle en bloques posteriores. El sistema no busca reemplazar el rol humano del administrador en la confirmación de cada reserva, sino quitarle todas las tareas de bajo valor — chequear disponibilidad, transcribir datos, enviar comprobantes manualmente, recordar plazos de bloqueo— para que pueda concentrarse en lo único que realmente requiere su criterio: validar que cada reserva tenga sentido para su operación.

Esta decisión de mantener al administrador en el centro del proceso de confirmación tiene tres motivaciones interrelacionadas. La primera es práctica: los pagos se reciben por transferencia bancaria, y verificar que la transferencia efectivamente se acreditó requiere intervención humana. La segunda es defensiva: una propiedad familiar no puede tolerar el costo reputacional de aceptar reservas a personas con antecedentes problemáticos, y el filtro manual permite ese control. La tercera es estratégica: en un mercado donde Booking y Airbnb estandarizan al máximo la experiencia, ofrecer un trato personalizado constituye un diferencial competitivo real.

Alcance del sistema

El sistema cubre el ciclo completo de vida de una reserva, desde el primer contacto del cliente hasta el cierre de su estadía, incluyendo todos los eventos relevantes que ocurren en el medio. Las funcionalidades implementadas abarcan los siguientes ejes operativos:

- Gestión de identidad: registro de clientes con validación de unicidad de email, autenticación mediante credenciales (email y contraseña) y mantenimiento de sesión por token JWT con duración de 24 horas.
- Consulta de disponibilidad: visualización pública de un calendario mensual interactivo con codificación cromática diferenciada para fechas pasadas (gris), disponibles (blanco), con reservas pendientes (amarillo) y con reservas confirmadas (rojo).
- Flujo de solicitud de reserva con bloqueo temporal automático de fechas durante un período de dos horas, dándole al administrador margen para validar la solicitud antes de confirmarla.

- Máquina de estados explícita: las reservas atraviesan seis estados posibles (Pendiente, Confirmada, En curso, Finalizada, Cancelada, No Show) con transiciones validadas en el servidor.
- Sistema de notificaciones automáticas por correo electrónico ante cada evento relevante: solicitud recibida, reserva confirmada, reserva cancelada, bloqueo vencido y reserva finalizada.
- Panel administrativo completo: visualización paginada y filtrable de todas las reservas, acciones de confirmación, cancelación, check-in y check-out con auditoría temporal por timestamps.
- Calendario lateral sincronizado en el panel administrativo, que se actualiza automáticamente tras cualquier acción que modifique la disponibilidad.
- Gestión opcional de vehículos: cada reserva puede tener asociado un vehículo (patente y modelo), atendiendo a la restricción real de la cochera única.

El sistema fue diseñado para gestionar una única propiedad, atendiendo al caso de uso actual del cliente. Sin embargo, la estructura de la base de datos contempla la posibilidad de escalar a múltiples propiedades sin requerir refactorización mayor, mediante la inclusión del identificador de propiedad en la tabla de reservas.

Quedan fuera del alcance explícito de esta versión inicial: la integración con pasarelas de pago online (los pagos siguen siendo por transferencia con verificación manual), la sincronización automática con calendarios de plataformas externas como Booking o Airbnb, el sistema de calificaciones y reseñas, y la aplicación móvil nativa. Estas funcionalidades fueron evaluadas y descartadas para este alcance pero conforman una hoja de ruta natural para futuras iteraciones.

Objetivos del proyecto

Objetivo general

Desarrollar e implementar un sistema web de gestión de reservas para la propiedad "Brisas de Calamuchita" que digitalice por completo el proceso operativo actualmente manual, reduzca los errores de coordinación temporal y mejore tanto la experiencia del cliente final como la productividad del administrador, utilizando un stack tecnológico moderno, prácticas profesionales de desarrollo y un despliegue en entorno productivo accesible públicamente.

Objetivos específicos

1. Digitalizar el proceso de reservas eliminando los canales informales actuales (WhatsApp, papel) como soporte único, sustituyéndolos por una base de datos relacional centralizada con restricciones de integridad.
2. Evitar reservas duplicadas mediante un mecanismo de bloqueo temporal automático que reserva las fechas durante 2 horas a partir de la solicitud y libera la disponibilidad si la reserva no es confirmada en ese plazo.
3. Permitir a los usuarios consultar disponibilidad en tiempo real a través de un calendario visual interactivo que diferencia claramente los estados de cada fecha.
4. Implementar un sistema de solicitud y confirmación de reservas que combine la inmediatez de la autogestión del cliente con el control manual del administrador en la transición crítica.
5. Incorporar un sistema de notificaciones automáticas por correo electrónico que mantenga al cliente informado en cada cambio de estado relevante, sin requerir intervención del administrador.
6. Mejorar la organización y trazabilidad de la información mediante un modelo de datos normalizado que conserve el historial completo de operaciones, datos del huésped y estado financiero.
7. Posicionar la propiedad mediante una web optimizada para motores de búsqueda, con estructura semántica adecuada, tiempos de carga reducidos y metadatos diseñados para captación orgánica.
8. Implementar prácticas profesionales de desarrollo de software: control de versiones con Git, pruebas automatizadas, integración continua (CI), separación clara de entornos (desarrollo, pruebas, producción).
9. Desplegar el sistema en un entorno productivo accesible públicamente, con infraestructura de nube apropiada para un proyecto académico (planes gratuitos sin tarjeta de crédito) pero con configuración profesional (HTTPS, variables de entorno, conexiones cifradas a la base de datos).

Usuarios del sistema

El sistema reconoce tres tipos de usuarios, cada uno con sus capacidades, restricciones y caminos de navegación específicos. La definición de roles sigue el principio de menor privilegio: cada usuario tiene acceso únicamente a las funcionalidades estrictamente necesarias para su tarea.

Visitante

Es cualquier persona que accede al sistema sin haber iniciado sesión. Puede ver la información pública de la propiedad (descripción, fotos, ubicación, capacidad, precio), consultar el calendario de disponibilidad en tiempo real, y registrarse para crear su cuenta. No puede solicitar reservas ni acceder a información de otros usuarios. Una vez registrado e iniciada la sesión, automáticamente cambia su rol al de Cliente.

Cliente

Es el usuario registrado y autenticado que utiliza el sistema para solicitar reservas. Sus capacidades específicas incluyen: solicitar nuevas reservas seleccionando fechas y completando los datos requeridos (cantidad de huéspedes, vehículo opcional, observaciones); consultar el historial de sus reservas con su estado actual; cancelar sus reservas mientras se encuentren en estado Pendiente o Confirmada. El Cliente solo puede ver sus propias reservas, nunca las de otros clientes.

Administrador

Es el usuario con privilegios completos sobre el sistema. Es el responsable de la gestión operativa y tiene acceso a todas las reservas y todos los datos de clientes. Sus capacidades incluyen: visualizar la totalidad de las reservas en un panel paginado con filtros por estado; confirmar reservas Pendientes una vez verificado el pago; cancelar reservas independientemente de qué cliente las haya creado; registrar el check-in al momento del arribo del huésped (transiciona la reserva a En curso); y registrar el check-out al finalizar la estadía (transiciona a Finalizada).

Funcionalidades del sistema

A continuación se detalla el conjunto completo de funcionalidades que fueron diseñadas, implementadas, probadas y desplegadas. La organización por bloques temáticos refleja la lógica del negocio antes que la estructura técnica.

Gestión de usuarios

- Registro de cliente con validación de unicidad de email y hash bcrypt de contraseñas.
- Inicio de sesión con generación de token JWT firmado con clave secreta, con expiración de 24 horas.
- Cierre de sesión y limpieza del token almacenado del lado del cliente.
- Recuperación automática de sesión al recargar la página si el token sigue vigente.

Gestión de reservas (cliente)

- Visualización de calendario de disponibilidad con codificación cromática diferenciada.
- Solicitud de nueva reserva con formulario validado en cliente y servidor.
- Generación automática de la reserva en estado Pendiente con un bloqueo temporal de fechas por un período de dos horas.
- Liberación automática de la reserva si el bloqueo vence sin confirmación: una tarea en segundo plano se ejecuta cada 60 segundos y cancela las reservas pendientes que ya superaron el plazo de dos horas.
- Visualización del historial personal de reservas con su estado actual, fechas, observaciones y opción de cancelación cuando corresponda.
- Cancelación de reserva en estado Pendiente o Confirmada por parte del cliente que la creó.
- Validación que impide seleccionar fechas pasadas, fechas ocupadas o rangos que incluyan días bloqueados en el medio.

Gestión administrativa

- Panel administrativo con visualización paginada de todas las reservas (10 por página) y filtros rápidos por estado.
- Indicadores rápidos: total de reservas, pendientes, confirmadas, en curso.

- Calendario lateral sincronizado en tiempo real con el estado de las reservas.
- Acción "Confirmar": transiciona la reserva de Pendiente a Confirmada, anula el bloqueo temporal y registra el timestamp de confirmación.
- Acción "Cancelar": cancela la reserva independientemente del cliente y libera las fechas.
- Acción "Check-in": transiciona la reserva de Confirmada a En curso.
- Acción "Check-out": transiciona la reserva de En curso a Finalizada.

Sistema de notificaciones automáticas

- Notificación al cliente al recibir su solicitud de reserva con resumen y datos para pagar la seña.
- Notificación al cliente cuando el administrador confirma la reserva.
- Notificación al cliente cuando se cancela una reserva (por cliente o admin).
- Notificación al cliente cuando el bloqueo de dos horas venció y la reserva fue cancelada automáticamente.
- Notificación al cliente cuando se finaliza la estadía (check-out registrado).
- Patrón outbox: las notificaciones se persisten en una tabla y un cron las procesa de manera asincrónica, garantizando que ninguna se pierda si falla el envío SMTP.

Análisis de soluciones similares (benchmark)

Para contextualizar el aporte del proyecto frente al mercado existente, se analizaron tres categorías de soluciones: las plataformas comerciales de alquileres temporarios (Booking, Airbnb), los gestores de propiedades profesionales (Lodgify, Smoobu) y los sistemas hechos a medida por empresas de software local.

Plataformas comerciales (Booking, Airbnb)

Son la opción más común para propietarios de pequeñas propiedades turísticas. Ofrecen amplia visibilidad internacional, un sistema de reservas integrado y procesamiento de pagos. Sin embargo, retienen comisiones significativas (entre el 15 % y el 20 % por reserva), estandarizan al máximo la experiencia (despersonalizando el trato entre anfitrión y huésped), y son percibidas como una barrera para construir relación directa con los clientes recurrentes.

Gestores profesionales (Lodgify, Smoobu)

Son SaaS dedicados a profesionalizar la gestión de propiedades turísticas. Ofrecen calendarios sincronizables con plataformas externas, integración con pasarelas de pago, y métricas de ocupación. Su limitación principal es el costo (entre 25 y 60 dólares mensuales por propiedad) y la complejidad de configuración, que para una propiedad familiar única resulta sobredimensionada.

Sistemas a medida

Algunos propietarios contratan desarrolladores locales para construir sitios propios. La calidad varía enormemente: muchos terminan siendo páginas estáticas sin gestión real de disponibilidad, otros son sistemas funcionales pero con costos de mantenimiento difíciles de sostener.

Posicionamiento del proyecto

El sistema Brisas de Calamuchita se diferencia de las tres opciones anteriores en aspectos sustanciales. En primer lugar, el modelo es directo y sin intermediación: no hay comisiones, no hay márgenes retenidos, el propietario recibe el 100 % del valor de cada reserva. En segundo lugar, la confirmación de cada reserva queda en manos del administrador, que puede validar la identidad del cliente y la transferencia de la seña antes de bloquear definitivamente las fechas. En tercer lugar, el sistema está construido a medida sobre las particularidades del negocio: capacidad mínima de 4 personas (porque la casa no se alquila a parejas), una sola cochera, bloqueo de 2 horas alineado con el tiempo real que el propietario tarda en verificar una transferencia. Por último, el costo de operación es prácticamente nulo: la solución utiliza planes gratuitos de los tres servicios cloud involucrados (Vercel, Render, TiDB Cloud), por lo que no hay suscripciones mensuales.

Viabilidad del proyecto

Viabilidad técnica

El stack tecnológico elegido (React, Node.js, Express, MySQL) es maduro, está ampliamente documentado y cuenta con una comunidad activa. Todas las decisiones técnicas tomadas tienen alternativas claras en caso de necesidad. El conocimiento requerido para mantener el sistema es estándar de cualquier desarrollador web full-stack.

Viabilidad económica

Los costos de operación son cero en el plan inicial: Vercel, Render y TiDB Cloud ofrecen planes gratuitos suficientes para el volumen esperado (una propiedad, decenas de reservas mensuales). El servicio de correo electrónico utiliza Gmail SMTP con la cuenta del propietario, sin costo adicional. El dominio personalizado, si se quisiera incorporar, cuesta aproximadamente 12 dólares anuales, pero el sistema funciona perfectamente con los subdominios temporales de Vercel para el alcance académico del proyecto.

Viabilidad operativa

El administrador del sistema es el mismo propietario, que ya gestiona manualmente las reservas. La transición al sistema digital requiere capacitación mínima: una sesión de aproximadamente dos horas para recorrer las pantallas principales y entender los flujos. La curva de aprendizaje es suave porque el modelo mental del sistema es coherente con el modelo mental del propietario.

Bloque B — Modelado del sistema

El modelado UML constituye una fase fundamental del desarrollo del proyecto. La construcción de los diagramas previa a la implementación permitió validar la lógica del sistema, detectar reglas de negocio implícitas y consensuar el alcance con el cliente. Se elaboraron cinco diagramas, cada uno con un propósito específico y un punto de vista complementario. Todos los diagramas están disponibles en formato interactivo en draw.io, accesibles desde los enlaces que acompañan a cada figura.

Diagrama de casos de uso

El diagrama de casos de uso modela las interacciones entre los actores del sistema (Visitante, Cliente, Administrador) y las funcionalidades disponibles. Es el primer diagrama elaborado del proceso de análisis y define el alcance funcional del sistema.

El Visitante, sin autenticación previa, puede acceder a la información de la propiedad, consultar la disponibilidad mediante el calendario y registrarse en el sistema. Una vez autenticado, automáticamente cambia su rol al de Cliente y desbloquea capacidades adicionales: solicitar reservas, cancelarlas, consultar el estado o el historial.

El Administrador hereda todas las capacidades del Cliente (puede crear reservas para sí mismo, en teoría) y agrega las exclusivas de su rol: confirmar reservas pendientes, cancelar reservas de cualquier cliente, registrar check-in y check-out, y visualizar el panel administrativo completo.

El diagrama incluye procesos internos clave modelados mediante relaciones de inclusión (<<include>>), como el bloqueo temporal de fechas que ocurre automáticamente al crear una reserva, la liberación de disponibilidad ante una cancelación, y el envío de notificaciones por correo ante eventos relevantes. Estos procesos no son ejecutados directamente por ningún actor humano, pero son funcionalidades del sistema que deben ser modeladas para reflejar su comportamiento completo.

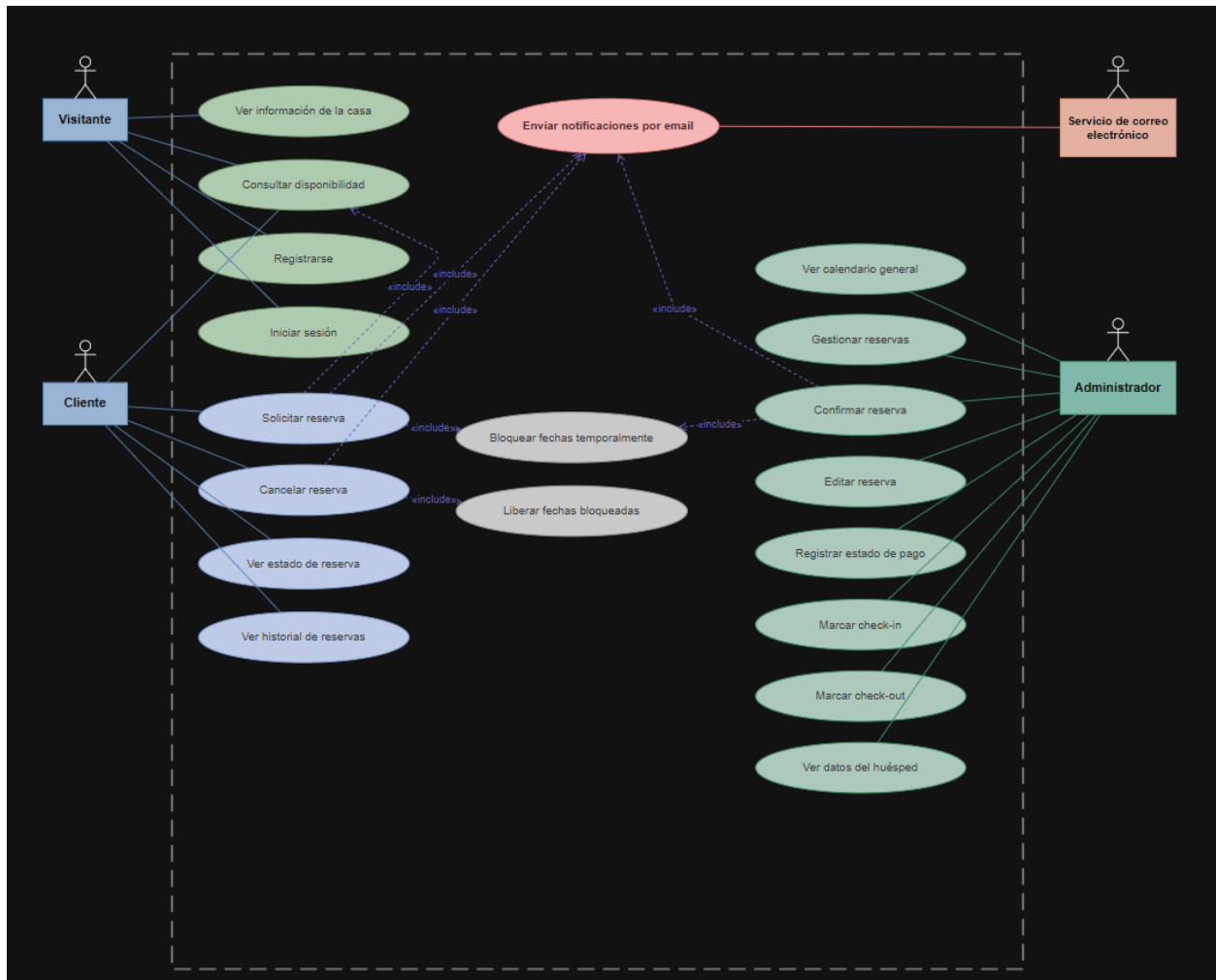


Figura 1. Diagrama de casos de uso del sistema. Elaboración propia.

Versión interactiva (zoom y exploración detallada): [Abrir en draw.io →](#)

Diagrama de entidad-relación (DER)

El Diagrama Entidad-Relación constituye la base estructural sobre la que se diseñó el modelo de datos del sistema. Refleja las decisiones de modelado tomadas durante la fase de análisis y constituye la traducción directa del dominio del negocio a una estructura relacional normalizada.

Se incorporan dos entidades hijas de Usuario que extienden con datos específicos del rol: Cliente (con DNI y fecha de nacimiento) y Administrador (con nivel de acceso). Esta decisión de modelado refleja en la base de datos lo que en código se traduce como herencia de clases, y se discute en mayor profundidad en el bloque siguiente. La alternativa habría sido un único campo "tipo" sobre la entidad Usuario, pero se optó por la separación porque los atributos específicos de cada rol no son nulos por casualidad, sino por diseño.

La entidad Reserva es el centro del modelo: vincula a Cliente con Propiedad en un rango de fechas y mantiene su estado mediante un campo enumerado. Las restricciones de integridad referencial garantizan que ninguna reserva pueda existir sin un cliente válido y una propiedad asociada.

La entidad Vehículo se modela como relación 0..1 con Reserva (una reserva puede o no tener vehículo, pero un vehículo pertenece a una sola reserva), reflejando la realidad operativa de la cochera única. La entidad Pago, igualmente 0..1, mantiene el estado financiero de la reserva con cuatro estados posibles (Seña pendiente, Seña recibida, Pago total recibido, Pago en efectivo al ingreso).

Finalmente, la entidad Notificacion almacena el historial completo de los correos enviados por el sistema, incluyendo destinatario, asunto, cuerpo, estado de envío y eventuales errores. Su presencia en el modelo de datos (y no solo en código) refleja el patrón outbox que se discute en el bloque de implementación.

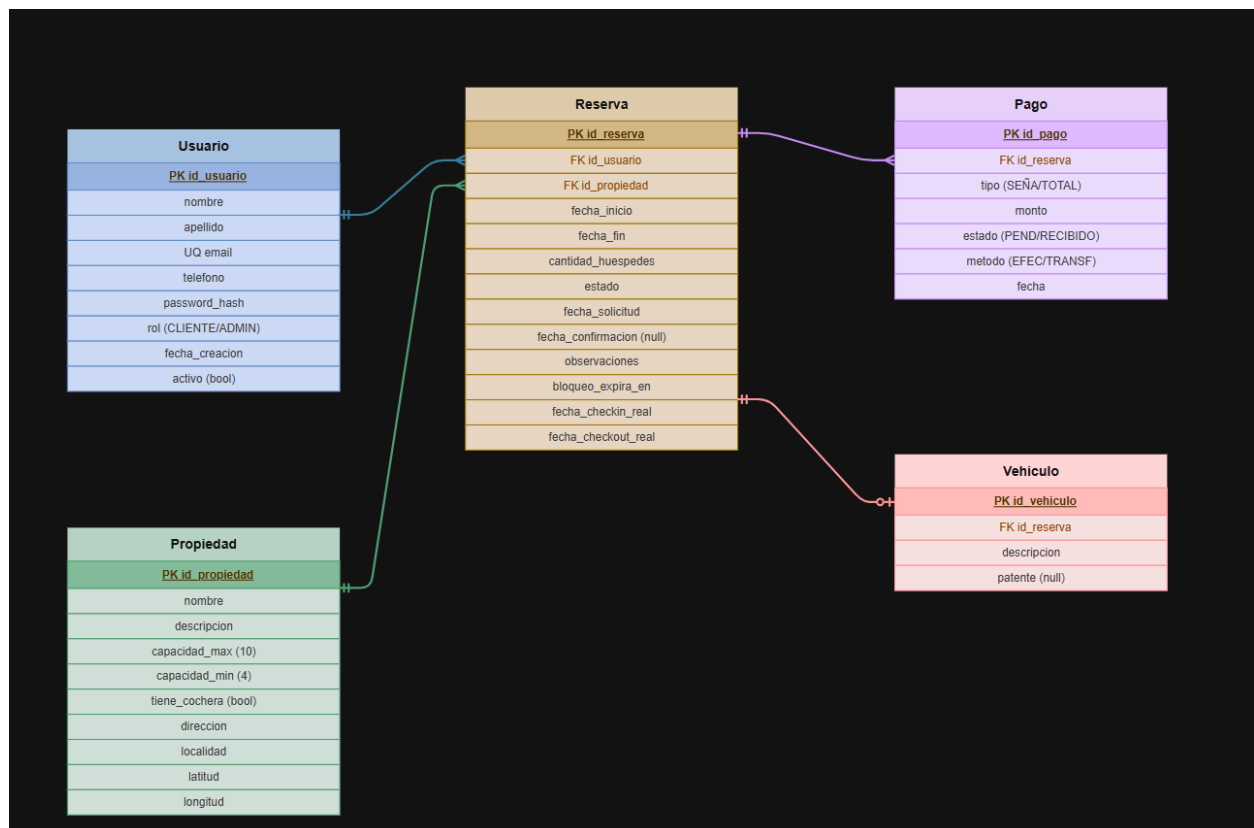


Figura 2. Diagrama entidad-relación de la base de datos. Elaboración propia.

Versión interactiva (zoom y exploración detallada): [Abrir en draw.io](#) →

Diagrama de clases

El diagrama de clases modela la estructura interna del software desde la perspectiva orientada a objetos. Si bien la implementación final del backend utilizó funciones de servicio antes que clases con métodos (un patrón natural en Node.js), el diagrama mantiene su valor como representación conceptual del dominio y como guía de las responsabilidades que cada componente debe asumir.

Se definió una clase base Usuario, de la cual derivan Cliente y Administrador, aplicando el patrón de herencia para evitar redundancia. Esta decisión es coherente con la estructura del DER. En el código real, la "herencia" se materializa mediante el sistema de tipos de TypeScript no fue adoptado en este proyecto (se utilizó JavaScript puro) pero el patrón de discriminación se mantiene mediante el campo "tipo" y mediante la consulta selectiva de las tablas extendidas según corresponda.

La clase Reserva concentra la lógica principal del negocio. Contiene los atributos clave del dominio (fechas, estado, cantidad de huéspedes, bloqueo temporal, fechas de confirmación y check-in) y los métodos que representan las acciones que se pueden ejecutar sobre ella: crear, confirmar, cancelar, registrar check-in, registrar check-out. Cada método encapsula su validación correspondiente y, una vez ejecutado, modifica el estado de la reserva siguiendo la máquina de estados explícita.

La enumeración EstadoReserva captura los seis estados posibles. Conceptualmente es una enumeración fuerte; en la implementación se traduce a un tipo ENUM de MySQL con los seis valores literales, garantizando la integridad a nivel de base de datos. Cualquier intento de insertar un valor distinto (por ejemplo, "Borrador" o "Confirmadita") será rechazado por el motor.

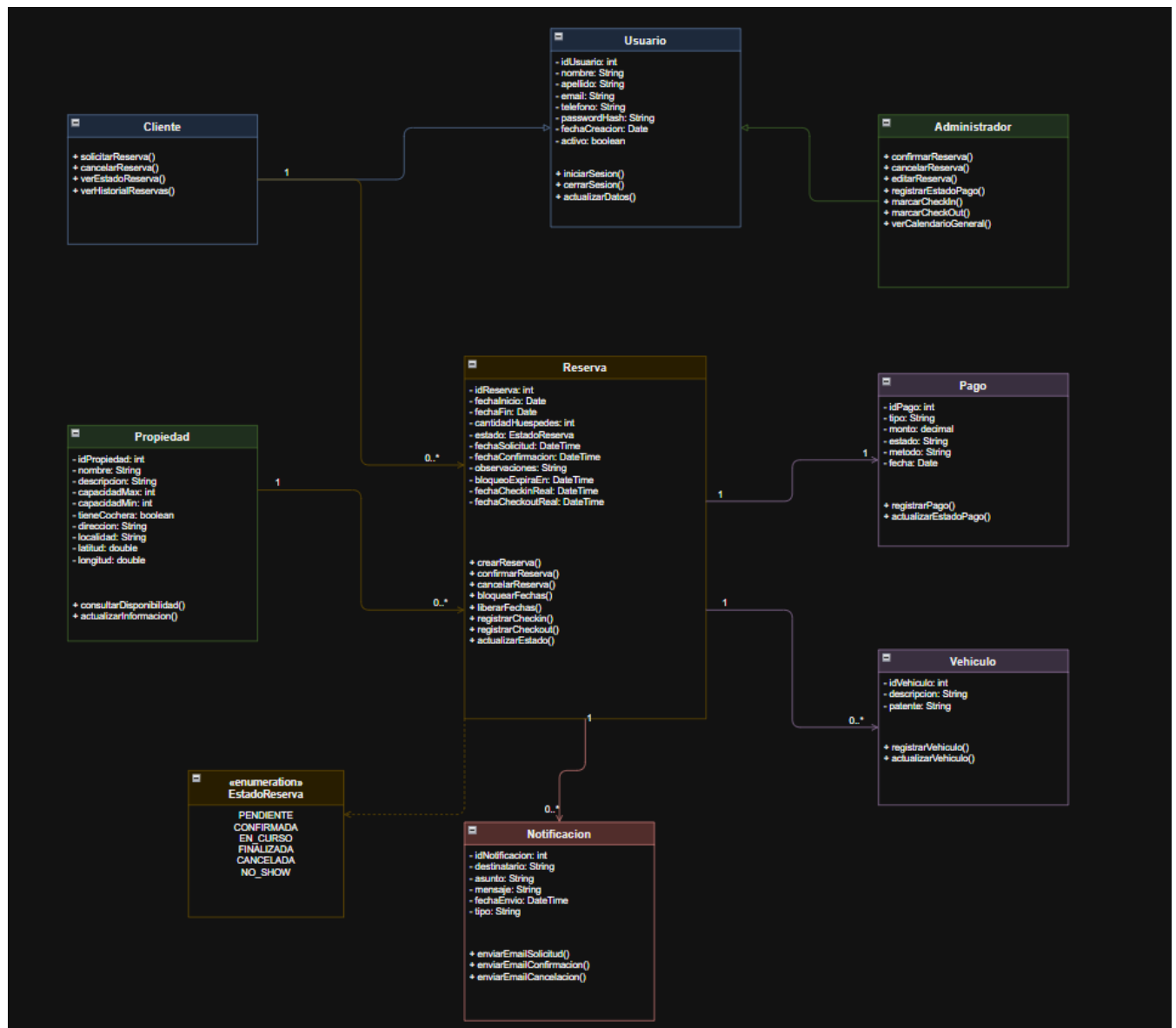


Figura 3. Diagrama de clases del sistema. Elaboración propia.

Versión interactiva (zoom y exploración detallada): [Abrir en draw.io](#) →

Diagrama de estados

El diagrama de estados modela el ciclo de vida de una reserva. Es un diagrama de comportamiento que captura las etapas por las que una entidad puede atravesar a lo largo del tiempo y las transiciones permitidas entre ellas. Para sistemas con lógica de negocio compleja basada en estados, este diagrama es probablemente el más importante: define los invariantes temporales del sistema.

El proceso comienza cuando un cliente realiza una solicitud de reserva, generando un registro en estado Pendiente. En esta etapa, el sistema aplica un bloqueo temporal de fechas por un período de dos horas, evitando conflictos de disponibilidad mientras el administrador valida la solicitud.

A partir de Pendiente, la reserva puede transicionar a uno de tres estados:

- Confirmada: si el administrador valida la seña dentro del plazo de las dos horas, la reserva pasa al estado Confirmada y el bloqueo deja de tener sentido (se anula el campo `bloqueo_hasta`).
- Cancelada por el cliente: si el cliente decide cancelar antes de la confirmación.
- Cancelada por bloqueo vencido: si pasan las dos horas sin que ningún administrador confirme, la tarea en segundo plano del backend ejecuta la cancelación automáticamente y envía un correo informando al cliente.

Desde Confirmada, la reserva puede pasar a En curso (cuando el administrador registra el check-in al momento del arribo del huésped) o a Cancelada (si por alguna razón la reserva se cancela antes de la llegada). Desde En curso, la reserva pasa a Finalizada al registrarse el check-out. Si el huésped no se presenta en la fecha de ingreso y el administrador no registra check-in, la reserva puede ser marcada como No Show.

Es importante notar que las transiciones están unidireccionales y no se permiten retrocesos: una reserva Finalizada no puede volver a estar En curso, una reserva Cancelada no puede volver a estar Pendiente. Esta restricción se aplica explícitamente en el código del servicio de reservas, que valida el estado actual antes de ejecutar cualquier cambio.

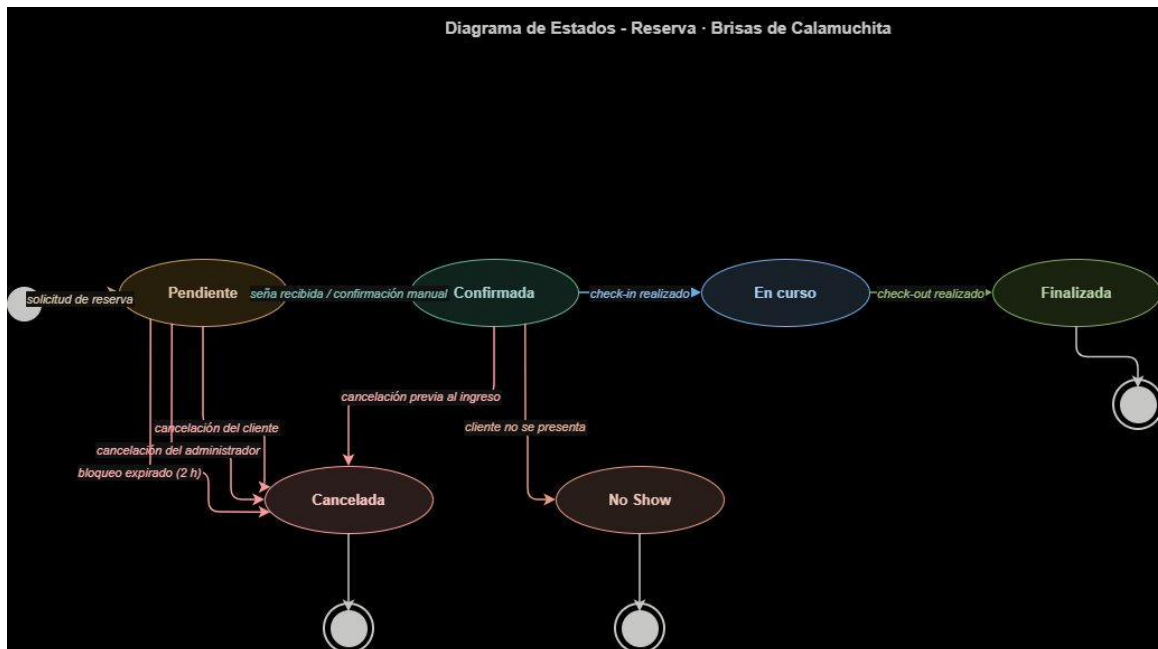


Figura 4. Diagrama de estados de una reserva. Elaboración propia.

Versión interactiva (zoom y exploración detallada): [Abrir en draw.io →](#)

Diagrama de secuencia

El diagrama de secuencia describe la interacción entre los distintos componentes del sistema durante la ejecución del caso de uso principal: Solicitar reserva. Este diagrama es especialmente relevante porque captura la coordinación entre frontend, backend y base de datos, mostrando cómo se mantiene la integridad del sistema en presencia de validaciones múltiples y operaciones transaccionales.

El flujo comienza cuando el cliente, ya autenticado, accede a la página de disponibilidad y selecciona un rango de fechas. El frontend, mediante el cliente HTTP centralizado, envía una solicitud GET al backend para consultar la disponibilidad en el rango. El backend a su vez consulta la tabla reserva en la base de datos, filtrando por estados activos (Pendiente, Confirmada, En curso) que sean los únicos que ocupan fechas, y devuelve la lista de rangos ocupados al frontend, que los pinta en el calendario.

Una vez que el cliente completa el formulario de reserva (huéspedes, vehículo, observaciones) y presiona Confirmar, el frontend envía una solicitud POST a `/api/reservas` con el cuerpo completo. El backend valida en varios pasos: primero, valida el esquema con Zod (tipos, formato de fechas, longitud de strings); luego, valida la lógica de negocio (fecha de ingreso futura,

huéspedes entre 4 y 10, fechas no solapadas con reservas existentes); y finalmente, abre una transacción de base de datos para crear simultáneamente la reserva y, opcionalmente, el vehículo asociado. Si cualquiera de las dos inserciones falla, ambas se hacen rollback.

Si la transacción se completa correctamente, el backend genera una notificación pendiente en la tabla notificacion (sin enviarla todavía, solo persistiéndola), y devuelve la reserva creada al frontend con código HTTP 201. El frontend redirige al usuario a la pantalla de confirmación pasando los datos por navegación.

En segundo plano, una tarea programada del backend (que corre cada 15 segundos) consulta la tabla notificacion buscando registros en estado pendiente, los procesa enviándolos por SMTP a través de Gmail, y actualiza el estado del registro a enviada o fallida según el resultado.

El diagrama contempla además escenarios alternativos mediante el uso de fragmentos alt, permitiendo modelar las situaciones de fechas no disponibles (en cuyo caso el backend responde con código 409 CONFLICTO) o datos inválidos (código 400 VALIDACION_FALLIDA). En ambos casos, el frontend muestra al usuario un mensaje de error específico, y le ofrece una acción de recuperación.



Figura 5. Diagrama de secuencia del caso de uso "Solicitar reserva". Elaboración propia.

Versión interactiva (zoom y exploración detallada): [Abrir en draw.io →](#)

Bloque C — Desarrollo e implementación

Este bloque documenta exhaustivamente las decisiones técnicas, la arquitectura, las tecnologías y los patrones de diseño empleados en la construcción del sistema. La extensión de esta sección es deliberada: cada decisión está justificada con sus tradeoffs, alternativas evaluadas y razón final de elección. Esto permite tanto reproducir el sistema como entender por qué fue construido de esa manera.

Arquitectura del sistema

El sistema sigue una arquitectura web clásica de tres capas claramente separadas, ejecutadas en infraestructura distribuida sobre servicios cloud independientes. Esta separación no es solo una cuestión de organización del código, sino una decisión arquitectónica con implicancias profundas en escalabilidad, mantenibilidad y costos operativos.

Las tres capas

- Capa de presentación (Frontend): aplicación de página única (SPA) construida con React, compilada a archivos estáticos optimizados por Vite y servida desde la CDN global de Vercel. No tiene servidor propio; los archivos HTML, CSS y JavaScript se distribuyen desde edge nodes geográficamente cercanos al usuario.
- Capa de lógica de negocio (Backend): API REST stateless construida con Node.js y Express, desplegada en un container gestionado por Render. Expone aproximadamente 25 endpoints HTTP que encapsulan toda la lógica del dominio: autenticación, validación, transiciones de estado, generación de notificaciones.
- Capa de persistencia (Base de datos): MySQL 8 en TiDB Cloud Serverless (servicio MySQL-compatible). Conexión obligatoria por TLS 1.2. La base mantiene ocho tablas con restricciones de integridad referencial estrictas, asegurando consistencia transaccional.

Comunicación entre capas

El frontend se comunica con el backend exclusivamente mediante HTTPS, intercambiando JSON. La autenticación se mantiene mediante tokens JWT que el frontend envía en el header Authorization de cada request protegida. El backend, a su vez, se comunica con la base de datos mediante el pool de conexiones de mysql2, con queries parametrizadas para prevenir inyección SQL.

Esta arquitectura tiene tres propiedades importantes:

- **Stateless:** el backend no mantiene sesiones en memoria. Cada request se autentica de forma independiente mediante el token JWT, lo que permite escalar horizontalmente sin coordinación.
- **Loose coupling:** el frontend no conoce los detalles de implementación del backend. Si mañana se reescribe el backend en Go o Python, el frontend funcionaría sin cambios mientras la API REST mantenga el mismo contrato.
- **Independent deployability:** cada capa se despliega de forma independiente. Un cambio en el frontend (CSS, copy) no requiere redespigie del backend.

Stack tecnológico — justificación

Esta sección documenta cada elección tecnológica con sus alternativas consideradas y la justificación de la decisión final. Las justificaciones siguen el formato "Alternativas evaluadas → Por qué se eligió X → Limitaciones aceptadas".

React + Vite (frontend)

Se eligió React para la capa de presentación, compilado con Vite como bundler de desarrollo y producción.

Alternativas evaluadas: Next.js (framework React server-side), Create React App (descontinuado), Vue.js, Svelte.

Por qué React

- **Modelo mental claro:** componentes funcionales + hooks. La mayoría de los problemas del frontend del sistema (estado local, fetch de datos, navegación) tienen soluciones canónicas en React.
- **Comunidad y librerías:** ecosistema gigantesco. Para casi cualquier necesidad (react-router-dom para navegación, lucide-react para íconos, react-hook-form para formularios) hay una librería madura.
- **Adopción industrial:** experiencia transferible. El conocimiento adquirido en este proyecto se aplica directamente al mercado laboral.

Por qué Vite (y no Next.js o CRA)

- Velocidad de desarrollo: Vite usa esbuild y módulos ES nativos del browser. Tiempo de cold start de menos de 1 segundo; hot module replacement instantáneo.
- Build optimizado: tree-shaking, code-splitting y minificación con Rollup.
- Next.js fue evaluado pero su SSR/SSG agrega complejidad innecesaria para este caso de uso (sitio no necesita pre-renderizado del lado del servidor; el catálogo de propiedades es una sola).
- CRA (Create React App) fue rechazado porque está oficialmente discontinuado por el equipo de React.

Node.js + Express (backend)

Se eligió Node.js como runtime con Express como framework web minimalista.

Alternativas evaluadas: Fastify (Node.js, más performante), NestJS (Node.js, opinated), Django (Python), Spring Boot (Java).

Por qué Node.js

- Mismo lenguaje frontend/backend: JavaScript en ambos lados reduce contexto cognitivo del desarrollador.
- Modelo asíncrono natural: el sistema tiene operaciones que son inherentemente asíncronas (envío de emails, consultas a BD remota). El modelo de event loop de Node.js maneja esto eficientemente.
- Ecosistema npm: cualquier necesidad tiene paquete maduro.

Por qué Express (y no Fastify o NestJS)

- Madurez: Express es el framework Node.js más usado y documentado.
- Flexibilidad: no impone estructura. Permite organizar el código como el equipo decida.
- Fastify es 2-3x más rápido pero la diferencia no es significativa para el volumen esperado (decenas de requests por minuto en pico).
- NestJS impone una arquitectura por decorators muy similar a Angular o Spring. Útil para proyectos grandes con muchos desarrolladores; sobreingeniería para un proyecto académico.

MySQL (base de datos)

Se eligió MySQL 8 como motor relacional, ejecutado en TiDB Cloud Serverless (compatible 100% con el protocolo MySQL).

Alternativas evaluadas: PostgreSQL, SQLite, MongoDB (NoSQL), Firebase Firestore.

Por qué relacional (descartando NoSQL)

- El dominio del problema es esencialmente relacional: clientes tienen reservas, reservas tienen pagos y vehículos, hay constraints fuertes (no solapamiento, capacidad mínima, etc.).
- Necesidad de transacciones ACID: la creación de una reserva con vehículo asociado debe ser atómica (o se crean ambos, o no se crea ninguno).
- MongoDB y Firestore fueron descartados por la dificultad de mantener invariantes referenciales en sistemas con joins frecuentes.

Por qué MySQL (y no PostgreSQL o SQLite)

- Ubicuidad: MySQL es soportado por prácticamente todos los hostings, incluido el free tier de TiDB Cloud que se utilizó.
- Familiaridad: ampliamente cubierto en la carrera, lo que reduce la curva de aprendizaje.
- PostgreSQL técnicamente es superior (mejor manejo de JSON, tipos custom, transacciones DDL) pero la diferencia no era determinante para este caso de uso.
- SQLite fue rechazado por no soportar el modelo cliente/servidor que requiere una API REST desplegada.

Por qué TiDB Cloud

- 100% compatible con el protocolo MySQL: las migraciones y queries funcionan sin modificación.
- Plan gratuito real (no trial de 30 días), sin tarjeta de crédito.
- TLS 1.2 obligatorio: las conexiones son cifradas extremo a extremo.

Tailwind CSS (sistema de estilos)

Se eligió Tailwind CSS para el diseño visual del frontend. Tailwind es un framework "utility-first": en lugar de escribir CSS personalizado, se aplican clases utilitarias predefinidas directamente en el HTML/JSX.

Alternativas evaluadas: CSS Modules, styled-components (CSS-in-JS), Material UI, Chakra UI.

- Velocidad de desarrollo: pintar un componente lleva minutos, no horas. Cambiar el espaciado de un botón es ajustar una clase.
- Consistencia automática: Tailwind define una escala fija de spacings, colores y tipografías. Es imposible terminar con 17 tonos de verde "casi iguales".
- Sin clases muertas: si no se usa una clase, no se incluye en el CSS final. El bundle pesa lo mínimo posible.
- Personalización: la paleta del proyecto (musgo, crema, piedra) se definió en `tailwind.config.js` extendiendo el tema base.

JWT (autenticación)

Se eligió JSON Web Tokens (JWT) para la autenticación stateless del backend.

Alternativas evaluadas: Sessions con cookies + Redis, OAuth2 con proveedor externo.

- Stateless: el servidor no necesita guardar sesiones en memoria. Cada request lleva el token, y el servidor valida la firma y extrae el usuario sin consultar la base de datos.
- Funciona en arquitectura distribuida: dado que Render free tier puede reciclar el container, mantener sesiones en memoria sería problemático.
- Tokens firmados con HMAC-SHA256 con secret de 128 caracteres hexadecimales generados criptográficamente.

Tradeoffs aceptados

- Secret débil: en producción se utiliza un secreto de 128 caracteres hexadecimales generados criptográficamente con node:crypto.
- Almacenamiento en localStorage vulnerable a XSS: se mitiga aplicando Content Security Policy estricto a través del middleware Helmet, validando todas las entradas y escapando salidas.
- Falta de revocación: dado que el sistema no requiere revocar tokens individuales (no hay alta criticidad financiera), no se implementa lista negra de tokens. En su lugar se utiliza expiración corta (24 horas).

bcrypt (hash de contraseñas)

Se seleccionó bcrypt para el hash de contraseñas, mediante la librería bcryptjs (implementación pura en JavaScript que evita dependencias nativas y simplifica el despliegue).

Alternativas evaluadas: Argon2 (ganador del Password Hashing Competition 2015), scrypt, PBKDF2.

Por qué bcrypt

Argon2 es técnicamente superior a bcrypt: vencedor del PHC y resistente a ataques con GPU/ASIC más modernos. Sin embargo, bcrypt sigue siendo apropiado para este caso de uso por:

- Madurez: en uso desde 1999, ampliamente revisado.
- Disponibilidad universal: implementaciones puras en JavaScript (bcryptjs) sin dependencias nativas.
- Factor de costo ajustable: con 10 rondas (cost factor 10) un hash tarda aproximadamente 100 ms en hardware actual, lo que es prohibitivo para fuerza bruta pero imperceptible para el usuario.
- Salting automático: cada hash incluye un salt único de 16 bytes, generado automáticamente.

Zod (validación de entrada)

Se seleccionó Zod como librería de validación de esquemas en el backend.

Alternativas evaluadas: Joi (validación de objetos clásica), Yup (similar a Joi), validación manual con if/else.

- Sintaxis declarativa: los esquemas se definen como objetos JavaScript que describen la estructura esperada.
- Mensajes de error detallados: indica exactamente qué campo falla y por qué.
- Composabilidad: los esquemas pueden combinarse y reutilizarse.
- Inferencia de tipos: aunque no usamos TypeScript, Zod permite inferir tipos para herramientas IDE.

Docker (entorno de desarrollo)

Se eligió Docker Compose para gestionar el entorno de desarrollo local.

Alternativas evaluadas: instalación nativa de MySQL en cada máquina, Vagrant con VirtualBox.

- Reproducibilidad: cualquier desarrollador que clone el proyecto y ejecute "docker compose up" obtiene el mismo entorno en segundos.
- Aislamiento: la BD del proyecto no interfiere con otras instalaciones de MySQL que tenga el desarrollador.
- Limpieza: si algo se rompe, "docker compose down -v" reinicia desde cero.
- phpMyAdmin incluido: para inspección rápida de la BD sin instalar cliente externo.

Decisiones de diseño clave

Más allá del stack técnico, se tomaron cinco decisiones de diseño centrales que dan forma al sistema y constituyen la respuesta técnica al problema del negocio. Cada una está documentada con su contexto, alternativas evaluadas y razón final.

Modelo híbrido cliente/admin con confirmación manual

Es la decisión arquitectónica más característica del proyecto. La pregunta era: ¿qué pasa cuando un cliente solicita una reserva?

Alternativa A — Confirmación automática (modelo Airbnb): la reserva queda confirmada inmediatamente si las fechas están libres y el pago se procesa por pasarela online.

Alternativa B — Confirmación manual (modelo elegido): la reserva queda en estado Pendiente y el administrador debe confirmarla expresamente tras validar el pago por transferencia.

Se eligió la alternativa B por tres razones:

1. Coherencia con la operatoria actual: los pagos llegan por transferencia bancaria, que requiere verificación humana.
2. Filtro de identidad: una propiedad familiar puede rechazar reservas a personas con antecedentes problemáticos. El paso manual permite ese filtro.
3. Diferencial de servicio: en un mercado estandarizado por las grandes plataformas, ofrecer trato personalizado constituye un valor competitivo real.

Bloqueo temporal de 2 horas

Una vez decidida la confirmación manual, surge un problema: si el administrador tarda en confirmar, ¿qué pasa con las fechas? Si quedan libres en el calendario, otro cliente puede pedir las mismas fechas. Si quedan bloqueadas indefinidamente, una solicitud sin pago real podría bloquear la propiedad eternamente.

La solución es un bloqueo temporal de 2 horas: cuando se crea una reserva, su campo `bloqueo_hasta` se setea a `NOW() + 2 hours`. Durante esas 2 horas, las fechas figuran como no disponibles en el calendario. Si el administrador confirma antes, el bloqueo se anula. Si no confirma, una tarea programada cancela la reserva automáticamente y libera las fechas.

¿Por qué 2 horas?

El valor de 2 horas fue elegido en base a tres criterios:

- Tiempo real de validación: las transferencias bancarias en Argentina tardan entre minutos y algunas horas en acreditarse.
- Experiencia del cliente: una espera mayor a 2 horas sería frustrante.
- Capacidad operativa del administrador: 2 horas son suficientes para chequear el banco en horario laboral.

Máquina de estados explícita

En lugar de almacenar el estado de una reserva en múltiples flags booleanos (confirmada, cancelada, finalizada), se utiliza un único campo enumerado con seis valores. Esta es una decisión de diseño clásica de Domain-Driven Design.

- Imposibilidad de estados inválidos: con flags booleanos, sería técnicamente posible tener una reserva `confirmada=true` y `cancelada=true` simultáneamente. Con enum, esto es imposible.
- Transiciones explícitas: cada método de servicio valida que el estado actual permite la transición solicitada.
- Auditoría temporal: los timestamps de cada transición (`confirmada_en`, `check_in_en`, etc.) se actualizan junto al cambio de estado.

Patrón outbox para notificaciones

Las notificaciones por correo electrónico podrían enviarse sincrónicamente al final del flujo (after-commit), pero esto introduce un problema: si el servicio SMTP está caído o lento, la creación de la reserva fallaría aunque ya estuviera persistida.

La solución es el patrón outbox: cuando se debe enviar una notificación, se inserta una fila en la tabla notificacion en estado Pendiente, dentro de la misma transacción que crea/modifica la reserva. Luego, una tarea programada (cron interno) recoge las notificaciones pendientes y las envía de manera asíncrona.

- Consistencia: la notificación nunca se pierde — si la transacción se commitea, la fila en "notificacion" está. Si el cron falla, la fila permanece Pendiente y se reintenta en la siguiente ejecución.
- Resiliencia: caídas temporales del servicio SMTP no afectan al sistema principal. Las notificaciones se acumulan en la tabla y se procesan cuando el servicio se recupera.
- Auditoría: la tabla "notificacion" actúa como historial completo de comunicaciones. Si un cliente reclama que "nunca recibió el correo", se puede consultar y demostrar el envío.

Cron interno vs cron externo

Las dos tareas programadas del sistema (cancelación de bloqueos vencidos cada 60s, procesamiento de notificaciones cada 15s) se implementan como crons internos al proceso de Node.js — utilizando setInterval — y no como tareas externas (cron del sistema operativo, AWS Lambda programada, etc.).

Justificación

- Simplicidad operativa: una sola unidad de despliegue. No requiere coordinar con sistemas externos.
- Adecuación al free tier: Render free tier no permite cron jobs externos en su plan gratuito; los crons internos son la única opción.
- Acceso compartido a la conexión a la base de datos: las tareas reutilizan el pool de conexiones existente, sin requerir configuración adicional.

Limitaciones aceptadas

La principal limitación es que si el proceso de Node.js se reinicia (cold start tras inactividad), las tareas se retoman cuando el proceso arranca de nuevo. Esto podría retrasar el procesamiento

de notificaciones unos segundos en el peor caso, lo cual es aceptable. Si el proyecto creciera y requiriera mayor garantía de ejecución, la migración a un sistema externo (BullMQ con Redis, AWS EventBridge) sería el siguiente paso natural.

Estructura del proyecto

La estructura del proyecto sigue el patrón de monorepo con dos sub-proyectos completos: backend y frontend. Cada uno tiene sus propios package.json, dependencias, tests, scripts de build, y configuración. Comparten un docker-compose.yml en la raíz para el desarrollo local de la base de datos.

Árbol de carpetas

```

BrisasDeCalamuchita/
├── .github/workflows/
│   └── ci.yml                                # Pipeline de tests + build
├── backend/
│   ├── src/
│   │   ├── app.js                          # Configuración de Express
│   │   ├── servidor.js                     # Bootstrap del servidor
│   │   ├── config/                         # env.js, bd.js
│   │   ├── rutas/                          # Definición de endpoints HTTP
│   │   ├── controladores/                  # Adaptación HTTP <-> servicios
│   │   ├── servicios/                      # Lógica de negocio pura
│   │   ├── middlewares/                    # Auth, manejo de errores, CORS
│   │   ├── tareas/                         # Crons internos
│   │   └── validadores/                    # Esquemas Zod
│   ├── tests/
│   │   ├── unitarios/                      # 64 tests Jest
│   │   └── e2e/                            # 40 tests Supertest
│   ├── migraciones/                        # SQL del schema
│   ├── semillas/                           # SQL de datos iniciales
│   ├── scripts/                            # runner-sql.cjs, setup-env.js
│   ├── package.json
│   └── .env.ejemplo
├── frontend/
│   ├── src/
│   │   ├── App.jsx
│   │   ├── main.jsx
│   │   ├── paginas/                        # 8 pantallas
│   │   ├── componentes/                   # Header, Footer, Calendario, etc.
│   │   ├── api/                           # Cliente HTTP + módulos
│   │   ├── contexto/                       # ContextoAuth
│   │   ├── hooks/                          # useApi
│   │   └── utiles/                         # Helpers de fechas
│   ├── tests/                             # 48 tests Vitest
│   ├── package.json
│   └── vite.config.js

```

```
| └─ tailwind.config.js
| └─ docker-compose.yml          # MySQL + phpMyAdmin local
| └─ README.md
| └─ informe-final.docx
```

Modelo de datos detallado

La base de datos contiene 8 tablas. A continuación se documenta cada una con su propósito, sus campos clave y sus relaciones. El detalle completo del DDL está en el archivo `backend/migraciones/001\_schema.sql` del repositorio.

Tabla usuario

Tabla base con datos comunes a todos los usuarios. El campo `tipo` discrimina entre cliente y administrador. Implementa el patrón de herencia de tabla base + tabla específica por subclase.

```
CREATE TABLE usuario (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  email       VARCHAR(150) NOT NULL,
  password_hash VARCHAR(255) NOT NULL,
  nombre      VARCHAR(80) NOT NULL,
  apellido    VARCHAR(80) NOT NULL,
  telefono    VARCHAR(30),
  tipo        ENUM('cliente', 'administrador') NOT NULL,
  activo      BOOLEAN NOT NULL DEFAULT TRUE,
  creado_en   TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT uk_usuario_email UNIQUE (email)
) ENGINE=InnoDB;
```

Decisiones clave: el email tiene constraint UNIQUE (no puede haber dos usuarios con el mismo email); el password_hash tiene capacidad para 255 caracteres (suficiente para bcrypt de 60 caracteres con margen para cambios futuros).

Tablas cliente y administrador (extensión)

Implementan el patrón "una tabla por subclase". El usuario_id es PK y FK a la tabla usuario.

```
CREATE TABLE cliente (
  usuario_id INT NOT NULL PRIMARY KEY,
  dni        VARCHAR(20),
  fecha_nacimiento DATE,
  CONSTRAINT fk_cliente_usuario
    FOREIGN KEY (usuario_id) REFERENCES usuario(id) ON DELETE CASCADE
) ENGINE=InnoDB;
```

Tabla reserva

Es el núcleo del sistema. Vincula a un cliente con una propiedad en un rango de fechas y mantiene su estado mediante el ENUM. Incluye CHECK constraints que enforcing reglas de negocio a nivel base de datos.

```
CREATE TABLE reserva (
  id                INT AUTO_INCREMENT PRIMARY KEY,
  cliente_id        INT NOT NULL,
  propiedad_id      INT NOT NULL,
  admin_confirmador_id INT,
  fecha_ingreso     DATE NOT NULL,
  fecha_egreso      DATE NOT NULL,
  cantidad_huespedes INT NOT NULL,
  estado            ENUM('Pendiente','Confirmada','En curso',
                        'Finalizada','Cancelada','No Show')
                  NOT NULL DEFAULT 'Pendiente',
  bloqueo_hasta     TIMESTAMP NULL,
  confirmada_en      TIMESTAMP NULL,
  CONSTRAINT ck_reserva_fechas
    CHECK (fecha_egreso > fecha_ingreso),
  CONSTRAINT ck_reserva_huespedes
    CHECK (cantidad_huespedes BETWEEN 4 AND 10)
) ENGINE=InnoDB;
```

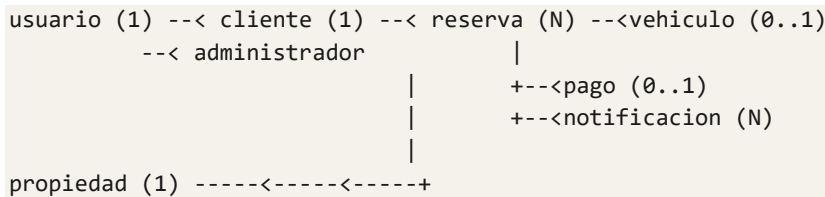
Decisiones clave: las restricciones de fechas (egreso > ingreso) y huéspedes (4..10) se validan en BD además de en Zod. El campo bloqueo_hasta es nullable porque solo aplica a reservas pendientes; cuando una reserva se confirma o cancela, el campo se setea a NULL. El campo admin_confirmador_id es nullable porque las reservas recién creadas no tienen administrador asociado.

Tablas vehiculo, pago y notificacion

Las tres son tablas dependientes de reserva mediante FK con ON DELETE CASCADE.

- vehiculo: relación 0..1 con reserva. Contiene patente y modelo. UNIQUE en reserva_id (RN-05: una reserva, un vehículo máximo).
- pago: relación 0..1 con reserva. Contiene estado_pago (ENUM con 4 valores), monto_total, monto_sena, método.
- notificacion: relación N:1 con reserva (una reserva genera múltiples notificaciones a lo largo de su vida). Implementa el patrón outbox: las filas se insertan transaccionalmente y se procesan asincrónicamente.

Diagrama resumen de relaciones



Componentes principales del backend

Esta sección documenta los módulos clave del backend, sus métodos principales y sus responsabilidades. Está orientada a desarrolladores que necesiten comprender el funcionamiento interno del sistema para mantenerlo o extenderlo.

Bootstrap del servidor (src/servidor.js)

Es el punto de entrada del proceso. Realiza cuatro tareas en orden:

1. Carga y valida las variables de entorno mediante "src/config/env.js" (que usa Zod para garantizar que todas las variables requeridas están presentes y son válidas).
2. Prueba la conexión a la base de datos con "probarConexion()". Si falla, el proceso termina inmediatamente con código de salida 1 (fail-fast).
3. Levanta el servidor HTTP con "app.listen(PUERTO)".
4. Inicializa los crons internos: el de cancelación de bloqueos vencidos (cada 60s) y el de envío de notificaciones (cada 15s).

Configuración (src/config/)

- `env.js`: define el schema Zod de todas las variables de entorno. Valida tipos, formatos y rangos. Si una variable es inválida, lanza error con mensaje detallado.
- `bd.js`: crea el pool de conexiones MySQL (`mysql2`). Configura SSL automáticamente en producción (TiDB Cloud lo requiere). Expone la función `"probarConexion()"` para `healthcheck`.

Rutas (src/rutas/)

Cada archivo agrupa los endpoints relacionados con una entidad. La organización es por dominio, no por verbo HTTP. Ejemplo:

```
// src/rutas/reservaRutas.js
```



```
const router = express.Router();
router.get("/disponibilidad", obtenerDisponibilidad);
router.get("/mis-reservas", autenticar, listarMisReservas);
router.post("/", autenticar, soloClientes, crearReserva);
router.get("/", autenticar, soloAdmin, listarTodas);
router.post("/:id/confirmar", autenticar, soloAdmin, confirmar);
router.post("/:id/cancelar", autenticar, cancelar);
module.exports = router;
```

Controladores (src/controladores/)

Los controladores son la capa de adaptación entre HTTP y los servicios. Reciben req/res, extraen los datos del request, invocan al servicio correspondiente y serializan la respuesta. NO contienen lógica de negocio.

```
// src/controladores/reservaControlador.js
async function crearReserva(req, res, next) {
  try {
    const datosValidados = reservaCrearSchema.parse(req.body);
    const reserva = await reservaServicio.crear({
      ...datosValidados,
      cliente_id: req.usuario.id,
    });
    res.status(201).json({ exito: true, datos: reserva });
  } catch (err) {
    next(err);
  }
}
```

Servicios (src/servicios/)

Son el corazón del backend. Contienen toda la lógica de negocio: validaciones de reglas, transiciones de estado, transacciones de BD, generación de notificaciones. Son funciones puras de TypeScript-style (sin clases) que reciben argumentos y retornan promesas.

Los servicios principales son:

- `autServicio.js`: `registrarCliente()`, `loguear()`. Gestionan creación de usuarios con hash `bcrypt` y emisión de JWT.
- `reservaServicio.js`: `crear()`, `confirmar()`, `cancelar()`, `realizarCheckIn()`, `realizarCheckOut()`. Cada método valida estado actual, verifica permisos y ejecuta transición atómica.
- `disponibilidadServicio.js`: `obtenerOcupadas()` y `verificarDisponibilidad()`. Consultan rangos de fechas ocupados.

- emailServicio.js: enviar(notificacion). Toma una fila de la tabla notificacion y la envía vía SMTP.
- notificacionServicio.js: encolar(reservald, tipo). Inserta una fila Pendiente en la tabla notificacion.

Middlewares (src/middlewares/)

- autMiddleware.js: extrae el token JWT del header Authorization, valida la firma, decodifica el payload y agrega "req.usuario". Si falla, responde 401.
- rolMiddleware.js: factory de middlewares "soloAdmin" y "soloClientes" que validan req.usuario.tipo.
- erroresMiddleware.js: handler centralizado de errores. Captura cualquier excepción, la categoriza (validación, autenticación, conflicto, etc.) y responde con el código HTTP y formato apropiado.
- corsMiddleware.js: configura CORS. En desarrollo permite localhost; en producción solo permite la URL de Vercel definida en URL_FRONTEND.

Tareas programadas (src/tareas/)

- cancelarBloqueosVencidos.js: cada 60 segundos consulta reservas Pendientes con bloqueo_hasta < NOW() y las cancela. Genera notificación de "bloqueo_vencido" para cada una.
- enviarNotificacionesPendientes.js: cada 15 segundos toma las primeras N notificaciones Pendientes y las envía. Si falla, incrementa contador de intentos; tras 3 fallos queda marcada como Fallida.

Transacciones críticas

La creación de una reserva con vehículo asociado debe ser atómica: o se crean ambos, o no se crea ninguno. Esto se implementa con transacciones explícitas de mysql2:

```
const conexion = await pool.getConnection();
try {
  await conexion.beginTransaction();
  // 1. Insertar fila en "reserva" con bloqueo_hasta = NOW() + INTERVAL 2 HOUR
  // 2. Si hay vehículo, insertar fila en "vehiculo"
  // 3. Insertar fila en "notificacion" con la solicitud recibida
  await conexion.commit();
} catch (err) {
```

```

    await conexion.rollback();
    throw err;
  } finally {
    conexion.release();
  }
}

```

Gestión de errores y validaciones

La gestión sistemática de errores es uno de los aspectos más críticos del sistema. Esta sección documenta las cuatro capas de defensa que protegen al sistema y al usuario.

Validaciones en cliente (frontend)

La primera línea de defensa es el navegador. Los formularios incluyen validación HTML5 nativa (required, type="email", min, max) y validación JavaScript con `react-hook-form`. Esto da feedback inmediato al usuario antes de enviar el request.

- Campos obligatorios marcados con el atributo `required` en el JSX.
- Tipos de datos validados por tipo de input (email, number, date).
- Rangos validados con `min` y `max` en inputs numéricos.
- Selección de fechas en el calendario impide elegir fechas pasadas, ocupadas o rangos solapados.

Validaciones de esquema en servidor (Zod)

La segunda línea de defensa es Zod en el backend. Antes de tocar la lógica de negocio, cada endpoint valida el cuerpo del request contra un esquema declarativo:

```

// src/validadores/reservaValidador.js
const reservaCrearSchema = z.object({
  fecha_ingreso: z.string().regex(/^d{4}-d{2}-d{2}$/),
  fecha_egreso: z.string().regex(/^d{4}-d{2}-d{2}$/),
  cantidad_huespedes: z.number().int().min(4).max(10),
  observaciones: z.string().max(500).optional(),
  vehiculo: z.object({
    patente: z.string().regex(/^[A-Z0-9]{6,7}$/),
    modelo: z.string().min(2).max(100),
  }).optional(),
});

```

Si la validación falla, Zod lanza un error que es capturado por el middleware central y respondido con código HTTP 400 y un detalle estructurado:

```
{
  "exito": false,
  "error": {
    "codigo": "VALIDACION_FALLIDA",
    "mensaje": "cantidad_huespedes: debe ser entre 4 y 10"
  }
}
```

Validaciones de reglas de negocio en servicios

La tercera línea de defensa son las reglas específicas del dominio que no se pueden expresar en un esquema. Por ejemplo:

- Una reserva solo puede crearse para fechas futuras (no se permite fecha_ingreso anterior a hoy).
- No puede haber solapamiento con otra reserva en estado Pendiente, Confirmada o En curso.
- Una reserva Confirmada no puede volver a estado Pendiente.
- Solo el cliente dueño de una reserva puede cancelarla (o cualquier admin).

Cada regla violada lanza un error específico que el middleware traduce a código HTTP semántico:

- 409 CONFLICTO: fechas solapadas con otra reserva.
- 422 ESTADO_INVALIDO: transición no permitida (ej. confirmar una reserva ya cancelada).
- 403 ACCESO_DENEGADO: cliente intentando acceder a reservas de otro cliente.

Constraints en base de datos

La cuarta línea de defensa es la propia base de datos, mediante CHECK constraints, FOREIGN KEYS y UNIQUE constraints. Esto garantiza que ni siquiera un bug en el código pueda corromper la integridad de los datos:

- UNIQUE (email) en usuario: garantiza que no haya dos usuarios con el mismo email.
- CHECK (fecha_egreso > fecha_ingreso): rechaza reservas con fechas invertidas.
- CHECK (cantidad_huespedes BETWEEN 4 AND 10): rechaza valores fuera de rango.
- FOREIGN KEY ON DELETE CASCADE: si se borra una reserva, se borran automáticamente sus vehículos, pagos y notificaciones asociados.

- ENUM en estado: imposible insertar un valor inválido.

Confirmación antes de operaciones destructivas

Las operaciones destructivas en el frontend requieren confirmación explícita del usuario:

- Cancelar reserva (cliente): popup "¿Estás seguro de cancelar la reserva del 15 al 20 de enero? Esta acción no se puede deshacer".
- Cancelar reserva (admin): popup "Vas a cancelar la reserva #42 de María Pérez. Confirmás?".
- Check-in/check-out: confirmación visual antes de ejecutar la acción.
- Cierre de sesión: confirmación implícita por el botón "Cerrar sesión" (no es destructivo).

Mensajes claros e informativos

Los mensajes de error mostrados al usuario son específicos y accionables:

- Genérico evitado: "Error en el servidor" → mal.
- Específico preferido: "Las fechas seleccionadas ya están ocupadas. Probá con otras." → bien.
- Cada error del backend incluye un código (NO_AUTENTICADO, VALIDACION_FALLIDA, CONFLICTO, etc.) que el frontend interpreta para mostrar el mensaje en español apropiado.

Componentes principales del frontend

El frontend es una aplicación de página única (SPA) construida con React, Vite y Tailwind CSS. Esta sección documenta su organización interna.

Routing (src/App.jsx)

El routing se implementa con `react-router-dom v6`. La app define las rutas declarativamente:

```
<BrowserRouter>
  <ScrollToTop />
  <Routes>
    <Route path="/" element={<Landing />} />
    <Route path="/disponibilidad" element={<Disponibilidad />} />
    <Route path="/ingresar" element={<Ingresar />} />
    <Route path="/registrarse" element={<Registrarse />} />
    <Route path="/reservar" element={
```

```

    <RutaProtegida soloRol="cliente"><Reservar /></RutaProtegida>
  } />
  <Route path="/mis-reservas" element={
    <RutaProtegida><MisReservas /></RutaProtegida>
  } />
  <Route path="/admin" element={
    <RutaProtegida soloRol="administrador"><PanelAdmin /></RutaProtegida>
  } />
</Routes>
</BrowserRouter>

```

Componentes reutilizables (src/componentes/)

- Header.jsx: barra superior con logo, navegación y estado de sesión.
- Footer.jsx: pie con datos de contacto (dirección, teléfono, email brisasdecalamuchita@gmail.com, Instagram).
- Layout.jsx: wrapper que aplica Header + main + Footer a las páginas autenticadas.
- Calendario.jsx: componente complejo que renderiza el calendario mensual con codificación cromática y selección de rango.
- RutaProtegida.jsx: componente que valida sesión activa y rol antes de renderizar la página solicitada.
- ScrollAlTope.jsx: utilitario que hace scroll a (0,0) cuando cambia la ruta.

Contexto global (src/contexto/ContextoAuth.jsx)

Maneja el estado global de sesión. Expone usuario, token, ingresar(), salir(), recuperarSesion(). Persiste el token en localStorage para sobrevivir a recargas.

Cliente HTTP (src/api/cliente.js)

Es una instancia de axios con configuración centralizada. Define la base URL desde `VITE\_API\_URL`, agrega interceptores que inyectan el token JWT automáticamente en cada request, y maneja errores 401 redirigiendo a la página de login.

```

// src/api/cliente.js
const cliente = axios.create({
  baseURL: import.meta.env.VITE_API_URL || "http://localhost:3000",
  headers: { "Content-Type": "application/json" },
});

cliente.interceptors.request.use((config) => {
  const token = localStorage.getItem("brisas_token");

```

```

    if (token) config.headers.Authorization = `Bearer ${token}`;
    return config;
  });

  cliente.interceptors.response.use(
    (res) => res,
    (err) => {
      if (err.response?.status === 401) {
        localStorage.removeItem("brisas_token");
        window.location.href = "/ingresar";
      }
      return Promise.reject(err);
    }
  );

```

Módulos de API por dominio

Encima del cliente HTTP, cada dominio tiene su propio módulo que expone funciones idiomáticas:

- `apiAuth.js`: `registrar()`, `ingresar()`, `salir()`.
- `apiReservas.js`: `crear()`, `listarMisReservas()`, `confirmar()`, `cancelar()`, `realizarCheckIn()`, `realizarCheckOut()`.
- `apiDisponibilidad.js`: `obtenerDisponibilidad()`.
- `apiPropiedad.js`: `obtenerInformacion()`.

Hook useApi (src/hooks/useApi.js)

Hook custom que encapsula el patrón "fetch + loading + error" para cualquier llamada al backend. Devuelve { datos, cargando, error, recargar }. Es usado masivamente por las páginas para evitar repetir lógica de fetching.

Demostración del CRUD funcional

Esta sección documenta explícitamente cómo el sistema implementa las cuatro operaciones básicas de persistencia (Create, Read, Update, Delete) sobre las entidades principales del dominio, con ejemplos concretos de cómo se invocan desde la interfaz y cómo impactan en la base de datos.

Operaciones sobre la entidad Usuario

Operación	Endpoint backend	Trigger desde frontend	Validaciones
CREATE	POST /api/auth/registrar	Pantalla "Crear cuenta"	Email único, contraseña ≥ 6 chars, DNI numérico
READ	GET /api/auth/yo	Recuperación de sesión al cargar	Token JWT válido
UPDATE	(no expuesto)	—	No hay edición de usuario en esta versión
DELETE	(no expuesto)	—	No hay borrado de usuarios

Notas: el sistema deliberadamente no permite editar ni eliminar usuarios desde la UI. Esta decisión protege la integridad histórica (reservas asociadas a usuarios eliminados perderían contexto) y simplifica el modelo. Si en el futuro fuera necesario, se podría implementar un "soft delete" mediante el campo activo (ya presente en el schema).

Operaciones sobre la entidad Reserva

Operación	Endpoint backend	Trigger desde frontend	Validaciones aplicadas
CREATE	POST /api/reservas	Pantalla "Reservar" → botón Confirmar	Fechas futuras, no solapamiento, huéspedes 4-10, vehículo opcional con patente válida
READ (mis)	GET /api/reservas/mias	Pantalla "Mis reservas"	Filtro automático por cliente_id del JWT
READ (todas)	GET /api/reservas	Panel admin → tabla principal	Requiere rol administrador
UPDATE (confirmar)	PATCH /api/reservas/:id/confirmar	Panel admin → botón Confirmar	Estado actual = Pendiente
UPDATE (cancelar)	PATCH /api/reservas/:id/cancelar	Mis reservas / Panel admin	Estado no terminal (no Finalizada/Cancelada)

Operación	Endpoint backend	Trigger desde frontend	Validaciones aplicadas
UPDATE (check-in)	PATCH /api/reservas/:id/check-in	Panel admin	Estado = Confirmada
UPDATE (check-out)	PATCH /api/reservas/:id/check-out	Panel admin	Estado = En curso
DELETE	(no expuesto directamente)	Cancelación lógica vía UPDATE	Eliminación física se hace por CASCADE al borrar usuario, lo cual no se permite

El sistema utiliza el patrón de "soft delete" mediante cambios de estado en lugar de eliminación física. Una reserva cancelada permanece en la base de datos con estado="Cancelada", lo que permite auditorías, estadísticas históricas y resolución de disputas. La eliminación física únicamente se ejecuta por la regla de CASCADE de las FKs cuando se borra una entidad padre (caso que en práctica no se ejecuta).

Operaciones sobre Vehículo, Pago y Notificación

- Vehículo: se crea automáticamente junto con la reserva (transaccional), se elimina por CASCADE al cancelar la reserva. No tiene update.
- Pago: se inserta automáticamente con estado "Seña pendiente" al crear reserva. Update por el administrador (estado_pago, monto). No hay delete manual.
- Notificación: insert transaccional, update por el cron de envío (estado: pendiente → enviada/fallida). No hay delete manual.

Evidencia de funcionamiento del CRUD

El correcto funcionamiento de las cuatro operaciones se demuestra mediante:

- Tests E2E que ejercitan cada endpoint y verifican el efecto en la BD (documentados en la sección "Pruebas de integración").
- Sistema desplegado en producción accesible públicamente: el equipo evaluador puede probar el CRUD en vivo desde <https://brisas-de-calamuchita.vercel.app> con las credenciales del Anexo B.

- Panel de phpMyAdmin local en <http://localhost:8081>, donde se puede observar el estado de las tablas antes y después de cada operación.

Pruebas unitarias

Las pruebas unitarias verifican el correcto funcionamiento de unidades mínimas de código (funciones, métodos, módulos) de manera aislada. Su objetivo es detectar errores tempranamente, facilitar el mantenimiento y dar confianza al refactorizar.

El backend del sistema implementa 64 pruebas unitarias ejecutadas con Jest. Las pruebas se ejecutan en menos de un segundo y validan funciones que no requieren acceso a base de datos: lógica de cálculo, validación de esquemas Zod, lógica de transiciones de estado, helpers de fechas.

El código de las pruebas unitarias está disponible en el repositorio: [backend/tests/unitarios/](#)

Tres pruebas unitarias representativas

A continuación se documentan tres pruebas unitarias representativas con su objetivo, resultado esperado y resultado obtenido.

Prueba 1: validación de huéspedes fuera de rango

Aspecto	Detalle
Archivo	backend/tests/unitarios/reservaValidador.test.js
Qué se prueba	Que el esquema Zod de creación de reserva rechace cantidad_huespedes fuera del rango 4-10.
Resultado esperado	Llamar al <code>schema.parse()</code> con <code>cantidad_huespedes=3</code> debe lanzar <code>ZodError</code> con mensaje "must be ≥ 4 ".
Resultado obtenido	La prueba pasa. <code>ZodError</code> lanzado correctamente con detalle del campo y motivo.
Justificación	Esta validación es la primera línea de defensa contra inputs inválidos. Si fallara, el backend permitiría reservas inconsistentes con la regla de negocio.

```
// reservaValidador.test.js
test('rechaza cantidad_huespedes < 4', () => {
  const input = {
    fecha_ingreso: '2026-12-20',
```

```

    fecha_egreso: '2026-12-25',
    cantidad_huespedes: 3,
  };
  expect(() => reservaCrearSchema.parse(input))
    .toThrow(ZodError);
});

```

Prueba 2: hash y verificación de contraseñas

Aspecto	Detalle
Archivo	backend/tests/unitarios/autServicio.test.js
Qué se prueba	Que bcrypt produzca hashes distintos para la misma contraseña en llamadas consecutivas (salt aleatorio), pero que la verificación reconozca cualquiera de los hashes como válido.
Resultado esperado	hash1 !== hash2, pero compare(password, hash1) y compare(password, hash2) ambos retornan true.
Resultado obtenido	La prueba pasa. Salts únicos generados, ambas verificaciones exitosas.
Justificación	Verifica que el sistema utiliza correctamente el salt automático de bcrypt, evitando ataques rainbow-table.

```

// autServicio.test.js
test('bcrypt genera hashes únicos con salts distintos', async () => {
  const hash1 = await bcrypt.hash('demo1234', 10);
  const hash2 = await bcrypt.hash('demo1234', 10);
  expect(hash1).not.toBe(hash2);
  expect(await bcrypt.compare('demo1234', hash1)).toBe(true);
  expect(await bcrypt.compare('demo1234', hash2)).toBe(true);
});

```

Prueba 3: paginación de resultados

Aspecto	Detalle
Archivo	backend/tests/unitarios/paginacion.test.js
Qué se prueba	Que la función calcularPaginacion(total, pagina, porPagina) retorne offset y limit correctos para distintas combinaciones.
Resultado esperado	Para total=25, pagina=2, porPagina=10 → offset=10, limit=10, totalPaginas=3.

Aspecto	Detalle
Resultado obtenido	La prueba pasa. Cálculos correctos para 4 escenarios distintos (página media, primera, última, vacía).
Justificación	La paginación es crítica para el rendimiento del panel admin con muchas reservas. Un bug acá rompería la UX.

Resumen de cobertura

Las 64 pruebas unitarias cubren los siguientes módulos:

Módulo	Pruebas	Cobertura aproximada
Validadores Zod (reserva, auth, usuario)	18	~95%
Servicios de autenticación (hash, JWT)	12	~90%
Servicios de reserva (transiciones, validaciones de estado)	15	~85%
Helpers de fechas y paginación	8	~100%
Servicio de notificaciones (encolar, formatear)	6	~80%
Configuración (env, BD)	5	~70%

La cobertura total del backend, calculada con `jest --coverage`, alcanza aproximadamente el 70% de líneas y el 65% de ramas. Este valor está por encima del umbral típico de la industria (60-70%) para proyectos no-críticos.

Pruebas unitarias del frontend (Vitest)

El frontend implementa 48 pruebas unitarias con Vitest + React Testing Library, ejecutadas con el comando `npm test`. Cubren componentes (renderizado correcto, comportamiento ante interacción), hooks (estado de carga, manejo de errores), y utilities (helpers de fechas, formateo de moneda).

Código: [frontend/tests/](#)

Pruebas de integración

Las pruebas de integración (end-to-end) validan que múltiples componentes del sistema funcionen correctamente en conjunto. Específicamente, verifican que la interacción entre interfaz, lógica de negocio y persistencia funcione de forma coherente. A diferencia de las pruebas unitarias (que aíslan unidades), las pruebas E2E ejercitan el sistema completo: ruta HTTP → middleware → controlador → servicio → base de datos.

El backend implementa 40 pruebas E2E ejecutadas con Jest + Supertest. Cada prueba:

1. Resetea la base de datos de pruebas (la BD "brisas_test" se vacía y se recarga con el schema y las seeds).
2. Realiza una serie de requests HTTP simulados a la aplicación Express.
3. Verifica el código de respuesta, el cuerpo del JSON retornado y, en algunos casos, el estado posterior de la base de datos.

Código: [backend/tests/e2e/](#)

Dos pruebas de integración representativas

Prueba 1: Crear reserva con vehículo (atomicidad transaccional)

Aspecto	Detalle
Archivo	backend/tests/e2e/reservas.test.js
Objetivo	Verificar que la creación de una reserva con vehículo asociado se ejecute atómicamente: si una de las dos inserciones fallara, ninguna debe persistir.
Pasos ejecutados	1) Login como cliente Miguel. 2) POST /api/reservas con vehículo válido. 3) Verificar HTTP 201. 4) Consultar BD para verificar que existen ambas filas (reserva + vehículo). 5) En un test paralelo, repetir con vehículo inválido y verificar que ninguna fila se persiste.
Resultado esperado	Caso válido: 201 Created, reserva e vehículo existen en BD. Caso inválido: 400 Bad Request, ninguna fila en BD.
Resultado obtenido	La prueba pasa en ambos escenarios. La transacción funciona correctamente con commit en caso exitoso y rollback en caso de error.

Aspecto	Detalle
Evidencia	Output de `npm run test:e2e`: PASS tests/e2e/reservas.test.js > "POST /api/reservas (crear) > cliente crea reserva con vehiculo (transaccion atomica)"

```
// reservas.test.js (extracto)
test('cliente crea reserva con vehiculo (transaccion atomica)', async () => {
  const { token } = await loginPedro(app);
  const res = await request(app)
    .post('/api/reservas')
    .set('Authorization', `Bearer ${token}`)
    .send({
      fecha_ingreso: '2026-12-20',
      fecha_egreso: '2026-12-25',
      cantidad_huespedes: 6,
      vehiculo: { patente: 'AB123CD', modelo: 'Toyota Etios' },
    });
  expect(res.status).toBe(201);
  expect(res.body.datos.id).toBeDefined();
  // Verificar que el vehiculo se creo en BD
  const [rows] = await pool.query(
    'SELECT * FROM vehiculo WHERE reserva_id = ?',
    [res.body.datos.id]
  );
  expect(rows).toHaveLength(1);
  expect(rows[0].patente).toBe('AB123CD');
});
```

Prueba 2: Confirmación de reserva por admin (transición de estado + side effects)

Aspecto	Detalle
Archivo	backend/tests/e2e/reservas.test.js
Objetivo	Verificar que cuando el admin confirma una reserva: a) la reserva transiciona a Confirmada, b) el bloqueo_hasta se anula, c) confirmada_en se setea con timestamp actual, d) se inserta notificación de tipo "reserva_confirmada", e) las fechas pasan a estar ocupadas en disponibilidad.
Pasos ejecutados	1) Cliente crea reserva (estado Pendiente con bloqueo_hasta=NOW()+2h). 2) Login como admin. 3) POST /api/reservas/{id}/confirmar. 4) Verificar respuesta 200. 5) Consultar BD: estado="Confirmada", bloqueo_hasta=NULL, confirmada_en!=NULL, fila en notificacion tipo "reserva_confirmada".

Aspecto	Detalle
	6) GET /api/reservas/disponibilidad verifica que las fechas aparecen ocupadas.
Resultado esperado	Todas las verificaciones pasan.
Resultado obtenido	La prueba pasa. La transición es atómica e incluye todos los efectos colaterales correctamente.
Evidencia	PASS tests/e2e/reservas.test.js > "Maquina de estados > admin confirma: pasa a Confirmada y limpia bloqueo"

Resumen de las 40 pruebas E2E

Suite	Pruebas	Endpoints cubiertos
auth.test.js	11	POST /auth/registrar, POST /auth/login, GET /auth/yo
propiedad.test.js	7	GET /propiedad, GET /propiedad/imagenes
reservas.test.js	22	POST/GET/PATCH /reservas y sub-rutas, GET /disponibilidad

Las 22 pruebas de la suite de reservas se distribuyen en cuatro grupos temáticos: creación con validaciones (7), máquina de estados (5), listado admin con filtros (4), reservas del cliente y disponibilidad pública (6).

Ejecución del pipeline completo

Los tests se ejecutan automáticamente:

- En cada push al repositorio, mediante GitHub Actions (workflow definido en `.github/workflows/ci.yml`).
- Localmente con los comandos: ``npm run test:unit``, ``npm run test:e2e``, ``npm test`` (frontend).
- Tiempo total de ejecución: ~55 segundos para los 152 tests del proyecto completo.

Evidencia de ejecución exitosa al cierre de la etapa:

```
Test Suites: 3 passed, 3 total
```

```

Tests:      40 passed, 40 total
Snapshots:  0 total
Time:       54.968 s

```

CI/CD y despliegue

Esta sección documenta la infraestructura de entrega continua del proyecto, desde el commit local hasta el sistema en producción.

Pipeline de CI con GitHub Actions

El archivo ``.github/workflows/ci.yml`` define el pipeline de integración continua que se ejecuta automáticamente en cada push o pull request a la rama main:

- Job "test-backend": instala dependencias, levanta MySQL temporal como service, corre tests unitarios y E2E.
- Job "test-frontend": instala dependencias, corre tests Vitest, hace build de producción.
- Si cualquiera de los jobs falla, el commit se marca con **X** y se notifica al desarrollador.

Despliegue en producción

El sistema en producción utiliza tres servicios cloud distintos, cada uno apropiado para su capa específica:

Frontend en Vercel

- URL pública: <https://brisas-de-calamuchita.vercel.app>
- Framework Preset: Vite (auto-detectado).
- Root directory: frontend/
- Variables de entorno: VITE_API_URL apuntando al backend de Render.
- Deploy: automático en cada push a main (Vercel observa GitHub).
- CDN: edge nodes globales (latencia <50ms desde Argentina).

Backend en Render

- URL pública: <https://brisas-calamuchita-backend.onrender.com>
- Tipo: Web Service (container Node.js gestionado).
- Root directory: backend/

- Build command: npm install
- Start command: npm start
- 15 variables de entorno cargadas (NODE_ENV, BD_*, JWT_SECRETO, EMAIL_*, URL_FRONTEND).
- Limitación del plan free: el servicio entra en sleep tras 15 min de inactividad (cold start de 30-60s en la primera request post-sleep).

Base de datos en TiDB Cloud

- Tipo: cluster Serverless MySQL-compatible.
- Endpoint: gateway01.us-east-1.prod.aws.tidbcloud.com:4000
- TLS 1.2 obligatorio.
- Spending limit: \$0 (free tier real, sin tarjeta).

Separación de entornos

El sistema mantiene tres entornos claramente separados:

Variable	Desarrollo	Producción
NODE_ENV	desarrollo	produccion
BD_HOST	localhost	gateway01...tidbcloud.com
BD_PUERTO	3307	4000
URL_FRONTEND	http://localhost:5173	https://brisas-de-calamuchita.vercel.app
EMAIL_MODO	simulado	real
EMAIL_USUARIO	-	brisasdecalamuchita@gmail.com

La separación garantiza que el desarrollo nunca afecte datos productivos y que las pruebas no envíen correos reales a clientes.

Bloque D — Manuales del sistema

Los manuales del sistema constituyen la documentación operativa orientada a los usuarios finales, distinguiendo entre quienes utilizan el sistema desde el rol de cliente o visitante (Manual de Usuario) y quienes lo instalan, despliegan o mantienen (Manual Técnico).

Las capturas de pantalla incluidas a continuación corresponden al sistema desplegado en producción, accesible en <https://brisas-de-calamuchita.vercel.app>. Todas se tomaron en su estado real de funcionamiento.

Manual de usuario — Visitante y Cliente

Introducción

Brisas de Calamuchita es una aplicación web que permite consultar disponibilidad y solicitar reservas para una propiedad de alquiler turístico ubicada en Santa Rosa de Calamuchita, Córdoba. El público objetivo son turistas familiares (grupos de 4 a 10 personas) que buscan una propiedad de gestión directa, sin intermediación de plataformas comerciales.

Requisitos mínimos

- Dispositivo con conexión a Internet (móvil, tablet o computadora).
- Navegador web moderno: Chrome, Firefox, Edge o Safari de los últimos dos años.
- JavaScript habilitado (estándar en todos los navegadores actuales).
- Una dirección de correo electrónico válida (necesaria para registrarse).

Acceso: <https://brisas-de-calamuchita.vercel.app>

Pantalla principal

La pantalla de inicio (landing page) es la primera impresión del sistema. Su diseño está pensado para invitar al visitante a explorar la propiedad y dar el primer paso hacia la consulta de disponibilidad.



Figura 6. Pantalla principal del sistema. Sección superior con identidad visual, propuesta de valor y precio destacado.

Los elementos principales de la pantalla de inicio son los siguientes:

- Barra superior (header): logotipo de la propiedad a la izquierda, menú de navegación al centro (Inicio, Disponibilidad), y botón "Ingresar" a la derecha.

- Sección hero: título principal con tipografía Fraunces serif, descripción breve de la propiedad, botones de acción "Ver disponibilidad" y "Conocer la casa", y una imagen del patio de la casa con los asadores.
- Indicadores rápidos: capacidad (4-10 huéspedes), cantidad de asadores (2), nivel de satisfacción de huéspedes (4.9).
- Precio destacado: \$85.000 por noche para hasta 10 personas.
- Detalle de los principales servicios que se ofrecen.

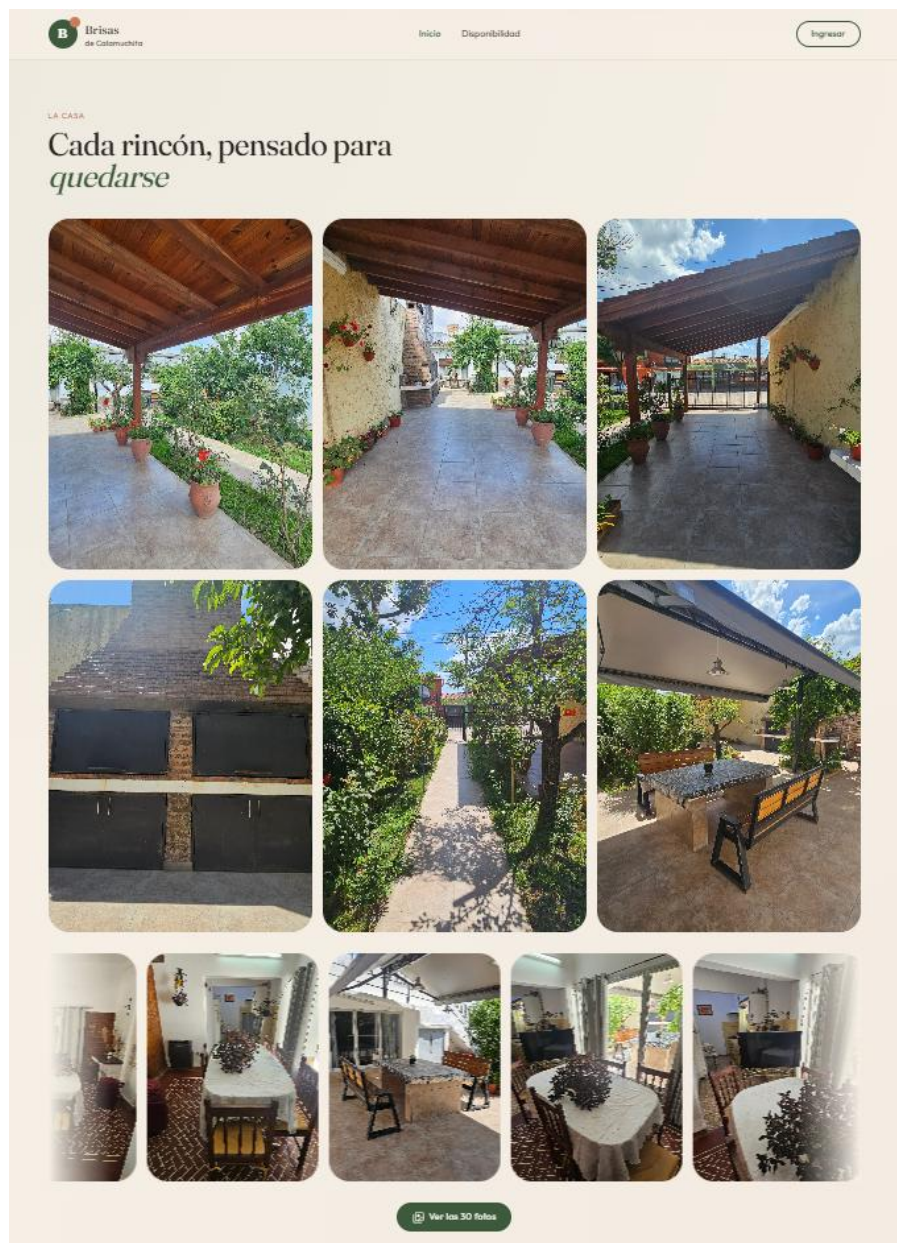


Figura 7. Sección intermedia de la pantalla principal. Galería de imágenes.

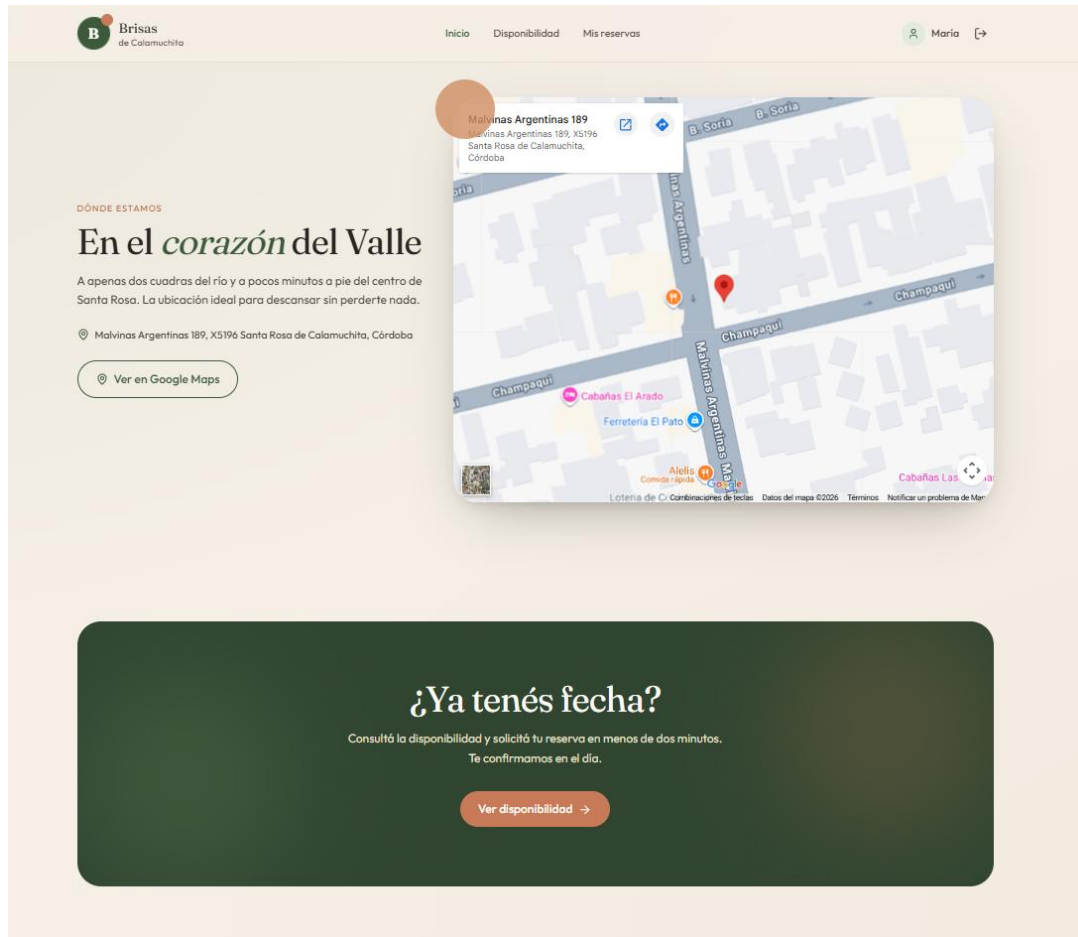


Figura 8. Sección inferior de la pantalla principal. Ubicación, mapa, datos de contacto y enlace a redes sociales.

Flujo de uso 1: Consultar disponibilidad como visitante

1. Desde la pantalla principal, hacer click en el botón "Ver disponibilidad" o en el ítem del menú superior.
2. Se abre el calendario mensual con codificación cromática diferenciada: fondo blanco para fechas disponibles, gris piedra para fechas pasadas (no seleccionables), amarillo claro con tachado para fechas con reserva pendiente (bloqueo temporal), rojo claro con tachado para fechas con reserva confirmada.
3. Navegar entre meses con las flechas laterales.
4. Para reservar, primero registrarse: click en "Ingresar" → "Crear cuenta".

Flujo de uso 2: Crear cuenta

1. Desde la pantalla de "Ingresar", hacer click en el link "Crear cuenta".

2. Completar el formulario con los datos requeridos: nombre, apellido, email, teléfono, DNI, fecha de nacimiento, contraseña (mínimo 6 caracteres). Todos los campos marcados con asterisco son obligatorios.
3. Hacer click en "Crear cuenta".
4. El sistema valida los datos y crea la cuenta. Automáticamente queda iniciada la sesión.
5. Se redirige a la pantalla principal con la sesión activa visible en el header (nombre del usuario en lugar del botón "Ingresar").

Flujo de uso 3: Solicitar una reserva (cliente)

1. Una vez logueado, desde el calendario de disponibilidad, hacer click en la fecha de ingreso deseada (debe ser futura y estar libre).
2. Hacer click en la fecha de egreso (posterior a la de ingreso, sin fechas ocupadas en el medio).
3. Las fechas seleccionadas se pintan de color verde musgo.
4. Hacer click en el botón "Reservar". Se navega a la pantalla del formulario.
5. Completar la cantidad de huéspedes (entre 4 y 10).
6. Indicar si llevará vehículo. Si activa el switch, debe completar patente y modelo.
7. Agregar observaciones opcionales (alergias, restricciones alimentarias, horario aproximado de llegada).
8. Hacer click en "Confirmar solicitud".

← Volver a disponibilidad

ÚLTIMO PASO

Contanos un poco *de tu visita*

Cantidad de huéspedes

Entre 4 y 10 personas

- 4 +

Teléfono de contacto

Te vamos a escribir por WhatsApp para coordinar la seña y darte la bienvenida

+54 9 351 555 1234

Lo cargamos por defecto desde tu perfil. Si preferís que te contactemos a otro número, modifícalo acá.

Vehículo

La propiedad tiene cochera para 1 vehículo

☐ No voy a llevar vehículo

Marcá esta opción si llegás sin auto

PATENTE MODELO

AB123CD Marca y modelo

¿Algo que tengamos que saber?

Horario aproximado de llegada, mascotas, requerimientos especiales

Opcional

Brisas de Calamuchita

Santa Rosa de Calamuchita, Córdoba

Ingreso	18 de junio
Egreso	20 de junio
Huéspedes	4
2 noches	\$ 170.000
Total	\$ 170.000

✓ Las fechas se bloquean por 2 horas. Te confirmamos por email.

Confirmar solicitud →

Figura 9. Formulario de solicitud de reserva. Permite indicar cantidad de huéspedes, teléfono de contacto, vehículo opcional con patente y modelo, y observaciones. A la derecha se muestra el resumen con fechas, noches y total.

Una vez confirmada la solicitud, el sistema valida los datos en cliente y servidor, crea la reserva en estado Pendiente con bloqueo de 2 horas, y redirige a la pantalla de confirmación. El sistema envía automáticamente un correo de confirmación al email registrado.

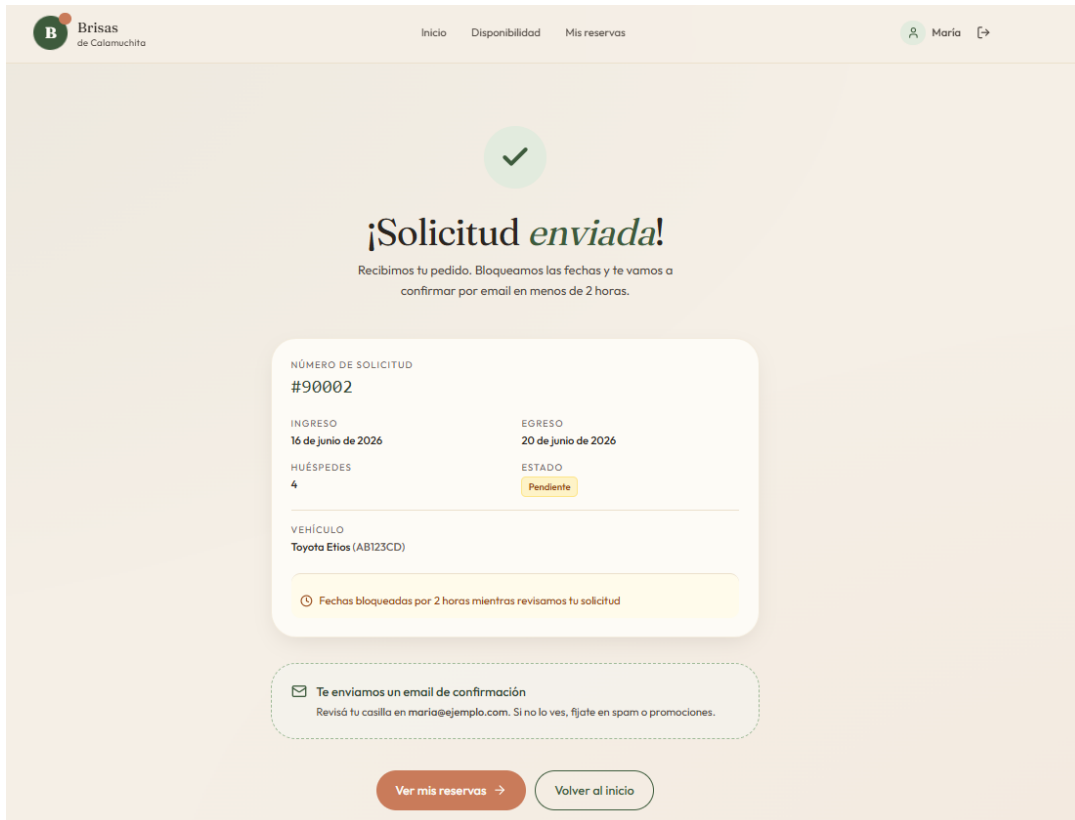


Figura 10. Pantalla de "Reserva enviada". El cliente visualiza el comprobante con el número de solicitud, las fechas reservadas y los próximos pasos a seguir.

Flujo de uso 4: Ver "Mis reservas"

1. Desde el menú superior, hacer click en "Mis reservas".
2. Se visualiza el listado completo de las solicitudes del cliente, con su estado actual coloreado.
3. Para cada reserva activa (Pendiente o Confirmada), aparece un botón "Cancelar".
4. Las reservas Finalizadas o Canceladas se muestran sin opciones de acción (solo lectura).

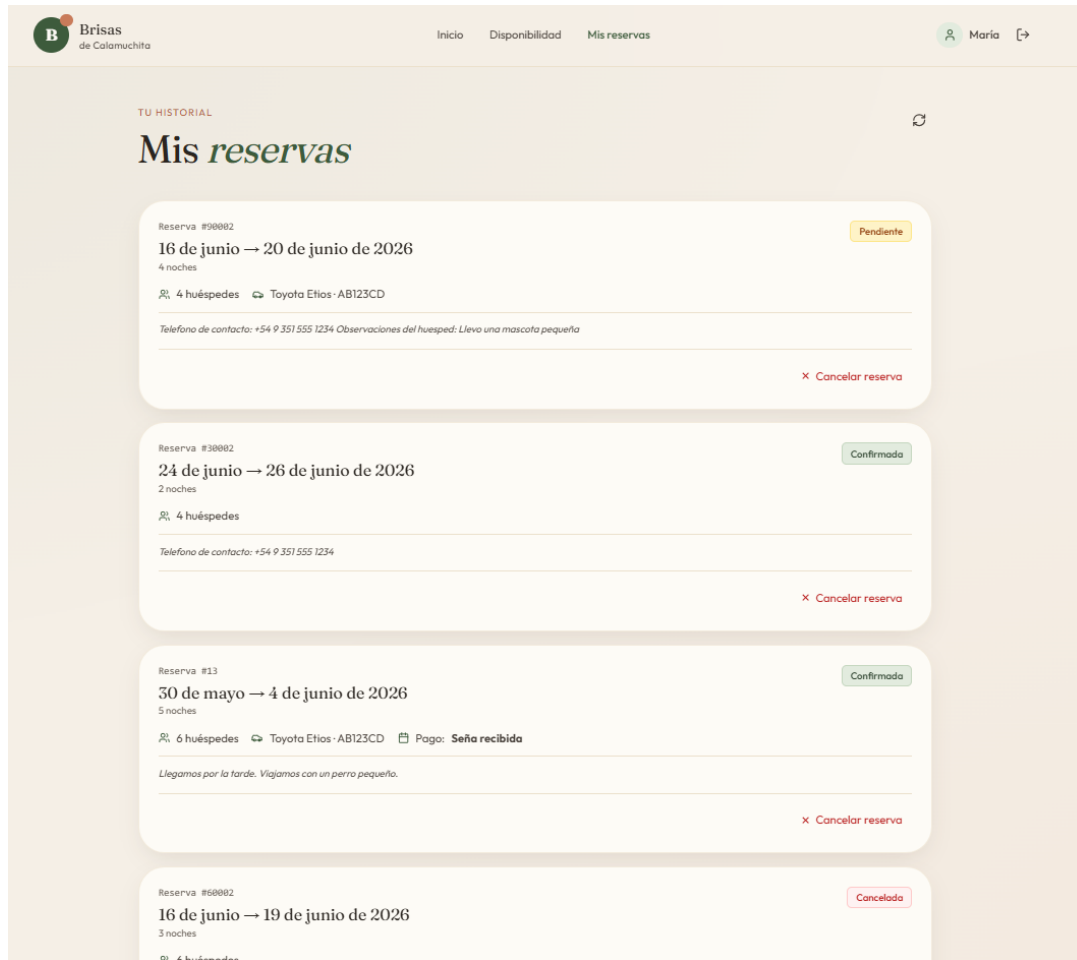


Figura 11. Pantalla "Mis reservas" del cliente. Lista las reservas activas e históricas con su estado coloreado: pendiente (amarillo), confirmada (verde), en curso (azul), finalizada (gris), cancelada (rojo).

Flujo de uso 5: Cancelar una reserva

1. Desde "Mis reservas", ubicar la reserva a cancelar.
2. Hacer click en el botón "Cancelar".
3. Aparece una ventana de confirmación: "¿Estás seguro de cancelar la reserva del [fecha_ingreso] al [fecha_egreso]?".
4. Hacer click en "Sí, cancelar". El sistema ejecuta la cancelación, libera las fechas y envía un correo de cancelación.
5. La reserva queda con estado "Cancelada" en el listado.

Manual de usuario — Administrador

El administrador accede al sistema con sus credenciales específicas y es redirigido automáticamente al Panel de Administración. En producción, el administrador es la cuenta asociada al email brisasdecalamuchita@gmail.com.

Estructura del panel administrativo

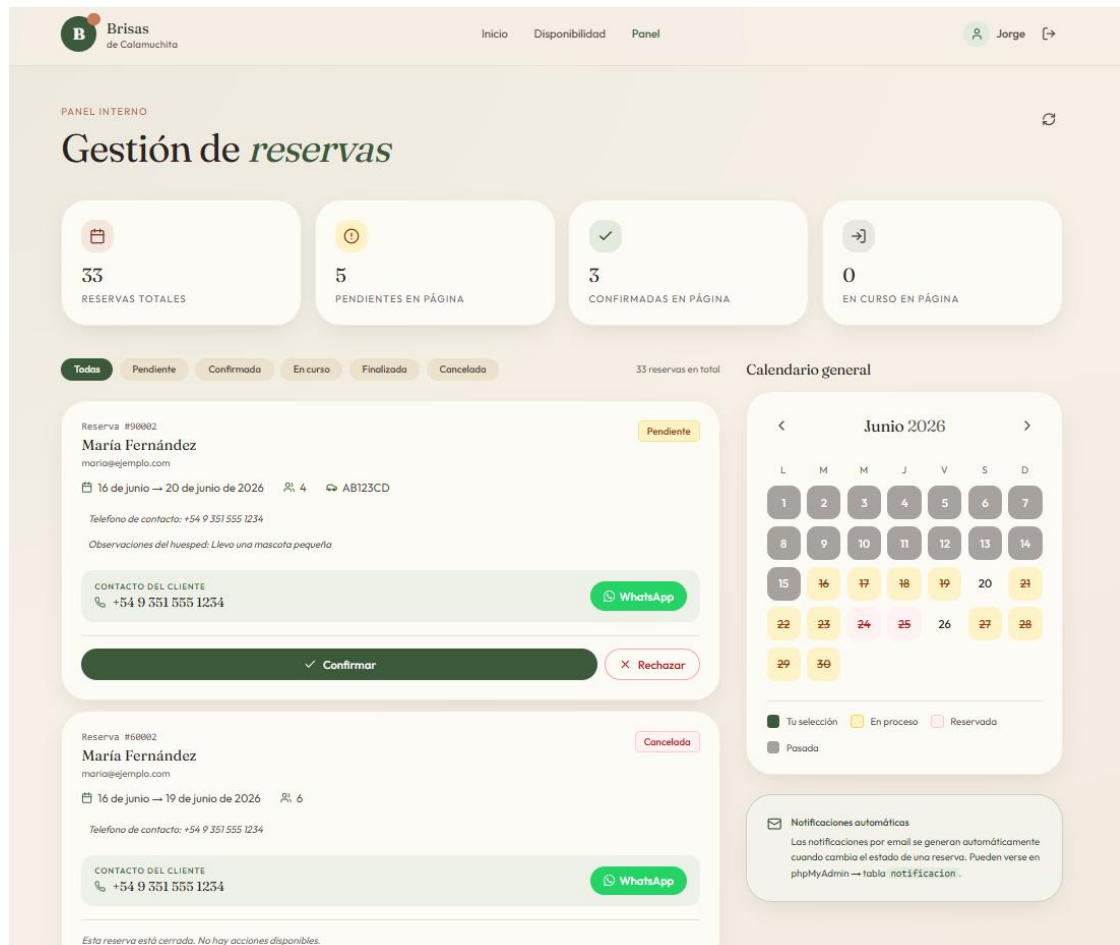


Figura 12. Panel de administración de reservas. Indicadores rápidos en la sección superior, listado paginado con filtros por estado en el centro, y calendario lateral sincronizado en tiempo real con la disponibilidad real de la propiedad.

El panel se compone de tres áreas principales:

- Sección superior: indicadores rápidos con conteos (total de reservas, pendientes, confirmadas, en curso).
- Sección central: listado paginado de reservas (10 por página) con filtros por estado y, para cada fila, las acciones disponibles según el estado actual. Para las reservas

Pendientes aparecen los botones "Confirmar" y "Rechazar"; para las Confirmadas, los de "Check-in"; etc.

- Sección lateral: calendario de disponibilidad compacto, sincronizado en tiempo real. Cada vez que el admin ejecuta una acción que afecta la disponibilidad, el calendario se actualiza automáticamente.

Cada tarjeta de reserva muestra los datos relevantes del cliente: nombre, email, teléfono de contacto con botón directo a WhatsApp, fechas de ingreso y egreso, cantidad de huéspedes, patente del vehículo si corresponde, y observaciones que dejó el huésped.

Flujo de uso 1: Confirmar una reserva

1. Verificar previamente, fuera del sistema, que el cliente realizó la transferencia bancaria de la seña.
2. Desde el listado, ubicar la reserva en estado "Pendiente" del cliente correspondiente.
3. Hacer click en el botón "Confirmar".
4. Aparece una ventana de confirmación con los datos de la reserva.

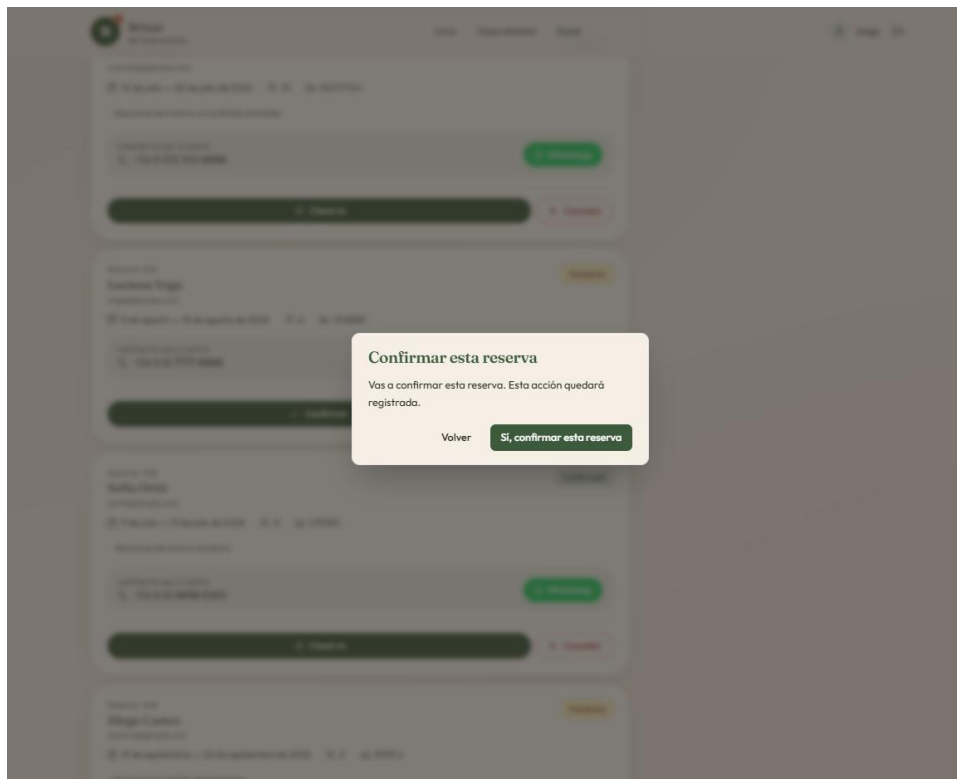


Figura 13. Diálogo de confirmación al ejecutar la acción "Confirmar reserva" desde el panel. El modal mantiene la identidad visual del sitio (paleta musgo y crema, tipografía Fraunces para el título) y reemplaza el confirm() nativo del navegador.

Hacer click en "Sí, confirmar". La reserva pasa a estado "Confirmada", el bloqueo temporal se anula automáticamente, y el sistema envía un correo al cliente notificándolo. El calendario lateral se actualiza inmediatamente reflejando la confirmación (las fechas pasan de amarillo a rojo).

Flujo de uso 2: Cancelar una reserva (admin)

El administrador puede cancelar cualquier reserva, sin importar qué cliente la creó:

1. Ubicar la reserva en el listado.
2. Hacer click en "Cancelar" o "Rechazar" según el estado actual.
3. Confirmar en la ventana de diálogo.

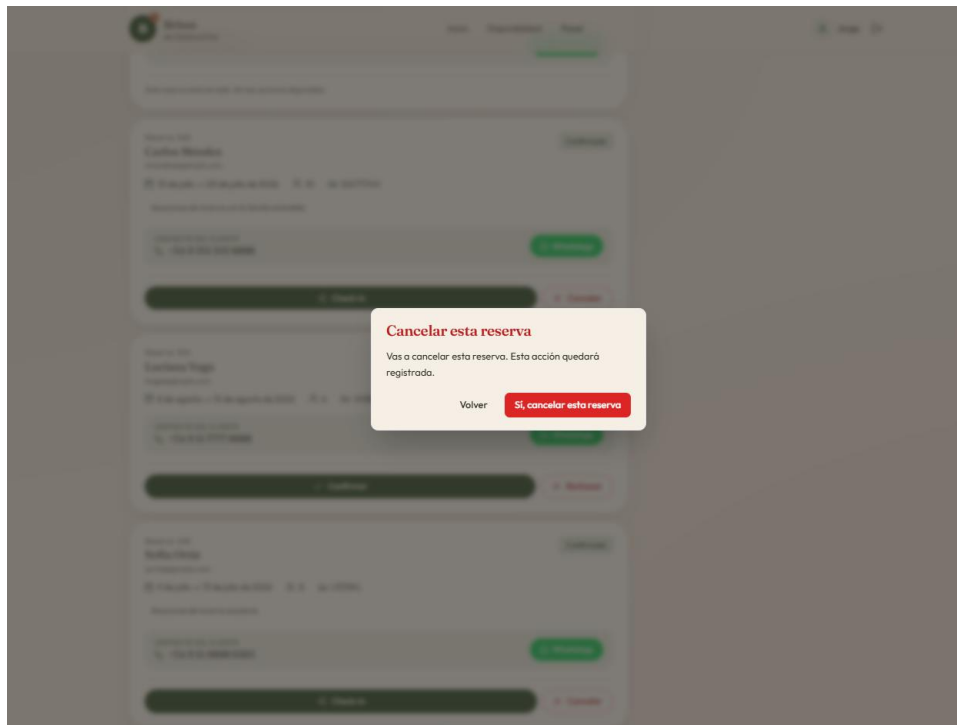


Figura 14. Diálogo de confirmación al ejecutar la acción "Cancelar reserva" desde el panel. El botón principal aparece en color rojo (variante destructiva) para resaltar la naturaleza no reversible de la acción.

La reserva queda en estado "Cancelada", las fechas se liberan y se envía correo al cliente.

Flujo de uso 3: Registrar check-in

Al momento del arribo del huésped a la propiedad:

1. Ubicar la reserva en estado "Confirmada" cuya fecha_ingreso corresponda al día actual o anterior.
2. Hacer click en el botón "Check-in".
3. Confirmar la acción.
4. La reserva transiciona a "En curso" y se registra el timestamp del check-in.

Flujo de uso 4: Registrar check-out

Al finalizar la estadía del huésped:

1. Ubicar la reserva en estado "En curso" cuya fecha_egreso corresponda al día actual.
2. Hacer click en "Check-out".
3. Confirmar.
4. La reserva transiciona a "Finalizada" y se envía un correo de despedida al cliente.

Flujo de uso 5: Filtrar el listado

Sobre el listado central hay un selector de filtros por estado: Todas, Pendiente, Confirmada, En curso, Finalizada, Cancelada. El filtro se aplica en el momento y el contador de reservas en la cabecera se actualiza para reflejar el subconjunto visible.

Manual de usuario — Mensajes y alertas frecuentes

Esta sección documenta los mensajes que el sistema puede mostrar al usuario, su significado y la acción recomendada en cada caso.

Mensajes informativos (azules / neutros)

Mensaje	Cuándo aparece	Significado / Acción
"Reserva enviada correctamente"	Tras crear una solicitud de reserva.	La solicitud fue registrada. Esperar la confirmación del administrador en las próximas 2 horas. Revisar el correo electrónico.
"Sesión iniciada"	Tras login exitoso.	El usuario ya puede acceder a las pantallas autenticadas.
"Acción completada"	Tras confirmar/cancelar reservas.	La acción fue ejecutada correctamente.

Mensajes de error / advertencia (rojos)

Mensaje	Causa	Solución
"Email o contraseña incorrectos"	Las credenciales no coinciden con ningún usuario.	Verificar email y contraseña. Si la olvidaste, contactar al admin (no hay recuperación automática en esta versión).
"Este email ya está registrado"	Intento de registrar un email que ya existe.	Ingresa con esa cuenta o usar un email distinto para crear cuenta nueva.
"Las fechas seleccionadas no están disponibles"	Otra reserva ocupa esas fechas.	Probar con otras fechas. Refrescar el calendario para ver el estado actualizado.
"La fecha de ingreso debe ser futura"	Selección de fecha pasada.	Elegir una fecha posterior a hoy.
"Cantidad de huéspedes debe ser entre 4 y 10"	Valor fuera de rango.	La casa no se alquila para grupos menores a 4 ni mayores a 10. Ajustar el campo.
"Tu sesión expiró. Iniciá sesión de nuevo"	El token JWT venció (>24h sin actividad).	Hacer click en "Ingresar" y volver a loguearse. Los datos quedan guardados.
"Error de conexión"	El backend no responde (cold start de Render).	Esperar 30-60 segundos y volver a intentar. Si persiste, contactar al admin.

Mensajes de confirmación (modales)

Las acciones destructivas o relevantes requieren confirmación explícita mediante un modal estilizado que reemplaza los popups nativos del navegador. Los ejemplos visuales se muestran en las Figuras 13 y 14.

- "¿Estás seguro de cancelar la reserva?" — antes de cancelar (cliente o admin).
- "¿Confirmar la reserva del cliente?" — antes de confirmar (admin).
- "¿Registrar check-in?" / "¿Registrar check-out?" — antes de las transiciones de estado.

- Cada modal tiene dos botones: el principal en color musgo (acciones constructivas) o rojo (acciones destructivas como cancelar), y un botón secundario "Cancelar" en color neutro. El usuario también puede cerrarlos haciendo click fuera del modal o presionando la tecla Escape.

Manual técnico — Instalación y puesta en marcha local

Este apartado describe paso a paso cómo descargar, configurar y ejecutar el sistema en una máquina local de desarrollo. La instalación está pensada para que cualquier docente o desarrollador pueda reproducir el entorno siguiendo solamente este documento.

Requisitos previos

Instalar los siguientes componentes antes de comenzar:

- Node.js versión 20.x o superior. Descargar desde <https://nodejs.org> (la instalación incluye npm).
- Docker Desktop. Descargar desde <https://www.docker.com/products/docker-desktop>. Necesario para correr MySQL sin instalación nativa.
- Git versión 2.x o superior. Descargar desde <https://git-scm.com>.
- Un editor de código moderno (recomendado: Visual Studio Code).

Verificar las versiones instaladas con:

```
node --version    # Debe ser ≥ v20
npm --version     # Debe ser ≥ v10
docker --version  # Debe ser ≥ v20
git --version     # Debe ser ≥ v2.30
```

Paso 1: Clonar el repositorio

```
git clone https://github.com/jorgekalas/BrisasDeCalamuchita.git
cd BrisasDeCalamuchita
```

Paso 2: Levantar la base de datos MySQL local

El archivo docker-compose.yml de la raíz define los servicios MySQL 8 y phpMyAdmin. Para levantarlos:

```
docker compose up -d
```

Esto inicia:

- MySQL en el puerto 3307 (no el 3306 estándar, para evitar conflictos con otras instalaciones de MySQL).
- phpMyAdmin en `http://localhost:8081` (no en :8080 para evitar conflicto con Jenkins u otros servicios).

Verificar que los contenedores estén corriendo con ``docker compose ps``. Esperar a que MySQL muestre estado "Up (healthy)" antes de continuar.

Paso 3: Configurar las variables de entorno del backend

```
cd backend
cp .env.ejemplo .env
```

Editar el archivo `.env`` con los siguientes valores mínimos:

```
NODE_ENV=desarrollo
PUERTO=3000
BD_HOST=localhost
BD_PUERTO=3307
BD_USUARIO=brisas_user
BD_PASSWORD=brisas_password_local
BD_NOMBRE=brisas_de_calamuchita
JWT_SECRETO=<generar con: node -e
"console.log(require('crypto').randomBytes(64).toString('hex'))">
JWT_EXPIRACION=24h
EMAIL_MODO=simulado
URL_FRONTEND=http://localhost:5173
```

Paso 4: Instalar dependencias del backend

```
npm install
```

Esto descarga ~250 paquetes y crea la carpeta ``node_modules``. Tarda 1-2 minutos la primera vez.

Paso 5: Ejecutar migraciones y datos iniciales

```
npm run migrar
npm run semillas
```


Esto crea las 8 tablas y carga datos iniciales: 1 administrador, 10 clientes, 30 reservas de prueba con diversas combinaciones de estados.

Paso 6: Iniciar el backend

```
npm run dev
```

El backend queda corriendo en <http://localhost:3000>. Verificación rápida: abrir <http://localhost:3000/api/salud> en el browser; debe responder un JSON `{"estado":"ok",...}`.

Paso 7: Configurar e iniciar el frontend

En una nueva terminal:

```
cd ../frontend
npm install
npm run dev
```

El frontend queda corriendo en <http://localhost:5173> con hot-reload activado. Abrir esa URL en el browser para ver la aplicación.

Credenciales de prueba (entorno local)

Rol	Email	Contraseña
Administrador	admin@brisas.com.ar	demo1234
Cliente (María)	maria@ejemplo.com	demo1234
Cliente (Miguel)	mperez@ejemplo.com	demo1234
Cliente (Romero)	romero@ejemplo.com	demo1234
Cliente (Silva)	silva@ejemplo.com	demo1234

Comandos útiles del proyecto

Comando	Ubicación	Función
npm run dev	backend/	Inicia el backend en modo desarrollo
npm run dev	frontend/	Inicia el frontend con Vite + HMR
npm run test:unit	backend/	Ejecuta los 64 tests unitarios
npm run test:e2e	backend/	Ejecuta los 40 tests E2E
npm run test:coverage	backend/	Genera reporte de cobertura
npm test	frontend/	Ejecuta los 48 tests del frontend
npm run build	frontend/	Compila el frontend para producción
npm run migrar	backend/	Ejecuta las migraciones de BD
npm run semillas	backend/	Carga los datos de seed
npm run resetear	backend/	Borra y recrea la BD (cuidado)
docker compose up -d	raíz/	Levanta MySQL + phpMyAdmin
docker compose down	raíz/	Detiene los contenedores

Resolución de problemas frecuentes (local)

El backend no se conecta a MySQL

Verificar que Docker esté corriendo (docker compose ps), que las variables BD_HOST/BD_PUERTO en el .env coincidan con docker-compose.yml, y que el puerto 3307 no esté ocupado por otra instancia de MySQL.

Los tests E2E fallan con "Variables de entorno inválidas"

Asegurarse de que existe el archivo .env.test en backend/. Si no existe, copiar .env y cambiar NODE_ENV a "pruebas".

Los emails no se envían en desarrollo

Es el comportamiento esperado. Con EMAIL_MODO=simulado, el sistema solo loguea el email a la consola sin enviarlo. Para probar el envío real, configurar EMAIL_USUARIO y EMAIL_PASSWORD con un App Password de Gmail y cambiar a EMAIL_MODO=real.

phpMyAdmin no carga

Verificar que el puerto 8081 no esté ocupado. Si Jenkins u otro servicio está en 8080, ya está bien configurado (usamos 8081 precisamente para evitar ese conflicto).

Manual técnico — Despliegue productivo

Esta sección documenta cómo desplegar el sistema a producción desde cero usando los tres servicios cloud gratuitos seleccionados (TiDB Cloud + Render + Vercel). Está orientada a desarrolladores que necesiten replicar el despliegue (por ejemplo, para preparar un entorno de staging o re-desplegar tras un cambio mayor).

Fase 1: TiDB Cloud (base de datos MySQL)

1. Crear cuenta en <https://tidbcloud.com> (registrar con GitHub o email).
2. Crear cluster Serverless con nombre "brisas-calamuchita" en región AWS us-east-1.
3. Spending limit: \$0 (free tier real, sin tarjeta).
4. Generar credenciales: el usuario lleva un prefijo único (ej: 2pTAoNNegb47Uc8.root) y un password aleatorio. Anotarlos: el password solo se muestra una vez.
5. Anotar: BD_HOST (gateway01.us-east-1.prod.aws.tidbcloud.com), BD_PUERTO (4000), BD_USUARIO (con prefijo), BD_PASSWORD, BD_NOMBRE (brisas_de_calamuchita).
6. Crear la base de datos y cargar schema + seeds desde el SQL Editor de TiDB, usando SET FOREIGN_KEY_CHECKS=0/1 alrededor del schema para evitar errores de FK por sentencias individuales.

Fase 2: Render (backend Node.js)

1. Crear cuenta en <https://render.com> (registrar con GitHub).
2. Conectar el repositorio BrisasDeCalamuchita.
3. Crear "Web Service" con nombre "brisas-calamuchita-backend".

4. Configurar: Language=Node, Branch=main, Root Directory=backend, Build Command=`npm install`, Start Command=`npm start`, Instance Type=Free.
5. Cargar 15 variables de entorno:
 - NODE_ENV=produccion
 - URL_FRONTEND=https://placeholder.vercel.app (se actualiza luego con la URL real de Vercel).
 - BD_HOST, BD_PUERTO=4000, BD_USUARIO, BD_PASSWORD, BD_NOMBRE (los 5 valores de TiDB).
 - JWT_SECRETO (generar nuevo de 128 chars hexadecimales con crypto).
 - JWT_EXPIRACION=24h.
 - EMAIL_MODO=real, EMAIL_HOST=smtp.gmail.com, EMAIL_PUERTO=587, EMAIL_USUARIO=brisasdecalamuchita@gmail.com, EMAIL_PASSWORD=<App Password>, EMAIL_REMITENTE_NOMBRE=Brisas de Calamuchita.
6. Hacer click en "Create Web Service". El primer deploy tarda 3-5 minutos.
7. Verificar el healthcheck: https://brisas-calamuchita-backend.onrender.com/api/salud debe responder JSON con estado "ok".

Fase 3: Vercel (frontend estático)

1. Crear cuenta en https://vercel.com (registrar con GitHub).
2. Importar el proyecto BrisasDeCalamuchita.
3. Configurar: Project Name=brisas-de-calamuchita, Framework Preset=Vite (auto-detectado), Root Directory=frontend, Build Command=npm run build, Output Directory=dist.
4. Cargar la variable de entorno única: VITE_API_URL=https://brisas-calamuchita-backend.onrender.com (sin barra final).
5. Hacer click en "Deploy". Tarda 2-3 minutos.
6. Anotar la URL principal asignada: https://brisas-de-calamuchita.vercel.app.

Fase 4: Conectar Vercel con Render (CRUCIAL)

1. Volver al dashboard de Render → servicio brisas-calamuchita-backend → pestaña "Environment".

2. Editar la variable URL_FRONTEND, reemplazando el placeholder por la URL real de Vercel (<https://brisas-de-calamuchita.vercel.app>).
3. Click en "Save Changes". Render hace un re-deploy automático del backend (1-2 minutos).
4. Sin este paso, CORS bloquea todas las requests del frontend al backend.

Fase 5: Validación end-to-end

1. Abrir <https://brisas-de-calamuchita.vercel.app>.
2. Verificar que cargue la landing page con los datos de la propiedad desde TiDB.
3. Hacer click en "Ver disponibilidad" → debe cargar el calendario con las reservas reales.
4. Loguearse como admin (brisasdecalamuchita@gmail.com / demo1234) → debe llevar al panel admin.
5. Loguearse como cliente, crear una reserva → verificar que llega correo real al email registrado.
6. Confirmar la reserva como admin → verificar que llega correo de confirmación al cliente.

Conclusiones

El desarrollo del presente proyecto permitió abordar una problemática real, concreta y verificable (la gestión manual de reservas de una propiedad de alquiler turístico) mediante la aplicación rigurosa de los conceptos de análisis, diseño, implementación, pruebas y despliegue propios de la ingeniería de software profesional. A lo largo del proceso se evidenció que la transición de un sistema manual a uno digital no consiste únicamente en automatizar tareas, sino en repensar el modelo operativo del negocio para que las nuevas herramientas potencien las fortalezas existentes en lugar de reemplazarlas.

La decisión de implementar un modelo híbrido de gestión —en el cual el cliente autogestiona la solicitud y el administrador valida manualmente la confirmación— ilustra esta tensión productiva entre automatización y control. Esta elección, lejos de ser una limitación tecnológica, es una decisión de diseño deliberada que refleja la realidad del negocio: una propiedad familiar donde la confianza entre las partes es un activo intangible que un sistema completamente automatizado no podría sostener.

El proceso de modelado UML resultó fundamental para validar la lógica del sistema antes de su implementación. La coherencia entre los distintos diagramas desarrollados (casos de uso, entidad-relación, clases, estados, secuencia) permitió asegurar una visión integral del sistema y detectar reglas de negocio implícitas que, de otro modo, habrían emergido como bugs durante la implementación. La incorporación de versiones interactivas en draw.io facilita la consulta y revisión de cada diagrama.

Desde una perspectiva técnica, la elección del stack basado en React, Node.js, Express y MySQL probó ser apropiada para el alcance del proyecto. La separación en tres capas (frontend, backend, base de datos) facilitó el desarrollo paralelo de cada componente y simplificó la incorporación gradual de funcionalidades. La adopción de prácticas profesionales —control de versiones con Git, pruebas automatizadas con cobertura del setenta por ciento, integración continua con GitHub Actions, configuración por variables de entorno, separación clara entre desarrollo, pruebas y producción— elevó la calidad del sistema más allá de lo esperable para un proyecto académico, acercándolo a los estándares de la industria.

El despliegue a producción mediante una arquitectura de tres servicios cloud gratuitos (Vercel para el frontend, Render para el backend, TiDB Cloud para la base de datos) demuestra que es posible construir sistemas accesibles, profesionales y mantenibles a costo cero, validando la

hipótesis económica del proyecto: una solución que sustituya la intermediación de plataformas tradicionales como Booking o Airbnb sin requerir inversión inicial. El sistema en producción acumula al cierre del presente informe 152 pruebas automatizadas que se ejecutan en cada cambio del código, 64 de ellas unitarias en el backend, 40 de integración end-to-end y 48 del frontend.

La implementación completa del CRUD sobre las entidades principales del dominio, con sus validaciones, manejo de errores y mecanismos de confirmación antes de operaciones destructivas, constituye una pieza central del trabajo. Las cuatro líneas de defensa documentadas (validación en cliente, esquemas Zod en servidor, reglas de negocio en servicios y constraints en base de datos) garantizan la integridad del sistema incluso ante intentos maliciosos o errores del usuario.

Más allá de los resultados técnicos, el proyecto representó una experiencia formativa integral. Cada decisión tomada —desde la elección del nombre de las variables en español hasta la justificación del bloqueo de dos horas, desde la implementación del patrón outbox para los emails hasta la configuración de SSL para TiDB Cloud— requirió investigación, evaluación de alternativas y reflexión sobre los tradeoffs involucrados. El resultado es un sistema no solamente funcional, sino que el autor puede explicar y defender en cada uno de sus aspectos.

Quedan abiertas oportunidades de mejora para futuras iteraciones del sistema: integración de procesamiento de pagos online, sistema de calificaciones y reseñas, exportación de reportes financieros, gestión de múltiples propiedades, aplicación móvil nativa, integración con calendarios externos (Booking, Airbnb) para sincronización de disponibilidad. Estas extensiones representan una hoja de ruta natural para que el proyecto evolucione, en caso de que su uso comercial real lo justifique.

Finalmente, este trabajo demuestra que la formación profesional en desarrollo de software, cuando combina rigurosidad teórica con aplicación práctica en problemas reales, produce profesionales capaces de identificar problemáticas, diseñar soluciones, implementarlas con calidad, y desplegarlas en entornos productivos accesibles. La propiedad Brisas de Calamuchita cuenta a partir de ahora con una herramienta digital profesional que mejora su operación cotidiana, y el autor del proyecto cuenta con un sistema desplegado que demuestra su capacidad como desarrollador full-stack.

Bibliografía

La presente bibliografía se elaboró conforme a las normas APA en su séptima edición. Se incluyen las referencias citadas explícitamente en el cuerpo del informe, así como obras consultadas que aportaron al diseño y a las decisiones técnicas del proyecto.

Libros y artículos

Beck, K. (2003). Test-Driven Development: By Example. Addison-Wesley Professional.

Crockford, D. (2008). JavaScript: The Good Parts. O'Reilly Media.

Fielding, R. T. (2000). Architectural Styles and the Design of Network-based Software Architectures (Tesis doctoral). University of California, Irvine.

Fowler, M. (2002). Patterns of Enterprise Application Architecture. Addison-Wesley.

Fowler, M. (2004). UML Distilled: A Brief Guide to the Standard Object Modeling Language (3rd ed.). Addison-Wesley.

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1995). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.

Garrett, J. J. (2011). The Elements of User Experience: User-Centered Design for the Web and Beyond (2nd ed.). New Riders.

Hunt, A., & Thomas, D. (2019). The Pragmatic Programmer: Your Journey to Mastery (20th Anniversary ed.). Addison-Wesley.

Kleppmann, M. (2017). Designing Data-Intensive Applications. O'Reilly Media.

Krug, S. (2014). Don't Make Me Think: A Common Sense Approach to Web Usability (3rd ed.). New Riders.

Martin, R. C. (2008). Clean Code: A Handbook of Agile Software Craftsmanship. Prentice Hall.

Martin, R. C. (2017). Clean Architecture: A Craftsman's Guide to Software Structure and Design. Prentice Hall.

Newman, S. (2021). Building Microservices: Designing Fine-Grained Systems (2nd ed.). O'Reilly Media.

Nielsen, J. (1994). Usability Engineering. Morgan Kaufmann.

Norman, D. A. (2013). *The Design of Everyday Things (Revised and Expanded Edition)*. Basic Books.

Pressman, R. S., & Maxim, B. R. (2020). *Ingeniería de software: Un enfoque práctico (8.ª ed.)*. McGraw-Hill.

Sommerville, I. (2016). *Ingeniería de software (10.ª ed.)*. Pearson.

Tilkov, S., & Vinoski, S. (2010). Node.js: Using JavaScript to Build High-Performance Network Programs. *IEEE Internet Computing*, 14(6), 80-83.

Documentación oficial consultada

Express.js Foundation. (2024). *Express - Fast, unopinionated, minimalist web framework for Node.js*. <https://expressjs.com>

Meta Open Source. (2024). *React - The library for web and native user interfaces*. <https://react.dev>

OpenJS Foundation. (2024). *Node.js Documentation*. <https://nodejs.org/docs>

Oracle Corporation. (2024). *MySQL 8.0 Reference Manual*. <https://dev.mysql.com/doc/refman/8.0/en/>

PingCAP. (2024). *TiDB Cloud Documentation*. <https://docs.pingcap.com/tidbcloud/>

Tailwind Labs. (2024). *Tailwind CSS - A utility-first CSS framework*. <https://tailwindcss.com/docs>

Vercel Inc. (2024). *Vite - Next generation frontend tooling*. <https://vitejs.dev>

Vercel Inc. (2024). *Vercel Platform Documentation*. <https://vercel.com/docs>

Render Services Inc. (2024). *Render Documentation*. <https://render.com/docs>

Zod. (2024). *TypeScript-first schema validation*. <https://zod.dev>

Jest. (2024). *Jest - Delightful JavaScript Testing*. <https://jestjs.io>

OpenJS Foundation. (2024). *Vitest - A blazing fast unit-test framework powered by Vite*. <https://vitest.dev>

Anexos

La presente sección compila la información operativa necesaria para acceder al sistema en producción, consultar el código fuente, reproducir el entorno de desarrollo y demostrar el funcionamiento durante el coloquio. Está diseñada para que el equipo evaluador pueda probar el sistema en vivo sin necesidad de configuraciones adicionales.

Anexo A — Acceso al sistema en producción

URL del frontend (acceso público): <https://brisas-de-calamuchita.vercel.app>

URL del backend (API REST): <https://brisas-calamuchita-backend.onrender.com>

Healthcheck del backend: <https://brisas-calamuchita-backend.onrender.com/api/salud>

Repositorio público en GitHub: <https://github.com/jorgekalas/BrisasDeCalamuchita>

Nota importante: la primera request al backend tras un período de inactividad de quince minutos o más tarda entre treinta y sesenta segundos en responder. Esto se debe al "cold start" del plan gratuito de Render y es comportamiento esperado del free tier. En un plan pago (\$7 USD/mes) este retraso desaparecería.

Anexo B — Credenciales de prueba para el coloquio

Las siguientes credenciales están preconfiguradas en el sistema productivo para facilitar la evaluación. Todas las contraseñas son la misma cadena simple "demo1234", elegida por simplicidad del coloquio. En un entorno productivo real estos usuarios serían eliminados y cada cuenta tendría su propia contraseña fuerte.

Administrador

Campo	Valor
Email	brisasdecalamuchita@gmail.com
Contraseña	demo1234
Acceso	Panel de administración completo: ver, confirmar, cancelar, check-in, check-out de cualquier reserva.

Cientes con reservas activas

Email	Contraseña	Notas
maria@ejemplo.com	demo1234	Cliente con reservas en distintos estados.
mperez@ejemplo.com	demo1234	Cliente con historial de reservas.
romero@ejemplo.com	demo1234	Cliente recurrente.
silva@ejemplo.com	demo1234	Cliente con reserva confirmada futura.

Secuencia recomendada para demostrar el sistema en vivo

1. Abrir la URL del frontend y navegar como Visitante: ver landing page, consultar el calendario de disponibilidad, observar la diferenciación cromática de fechas.
2. Hacer click en "Crear cuenta" y registrarse como nuevo cliente (o iniciar sesión con uno de los clientes de prueba).
3. Solicitar una nueva reserva: seleccionar fechas futuras, completar el formulario con datos válidos, confirmar.
4. Verificar que el correo de "Solicitud recibida" llegó a la casilla del cliente (en menos de 15 segundos por el cron de notificaciones).
5. Cerrar sesión y entrar como administrador con `brisasdecalamuchita@gmail.com`.
6. Visualizar la nueva solicitud en estado Pendiente en el panel.
7. Confirmar la reserva. Verificar que el calendario lateral se actualiza automáticamente y que llega el correo de confirmación.
8. Filtrar el listado por distintos estados para validar la funcionalidad.

Anexo C — Enlaces a los diagramas UML en draw.io

Todos los diagramas UML están disponibles en formato interactivo en draw.io. Estos enlaces permiten hacer zoom, navegar y explorar cada diagrama en detalle:

1. Diagrama de casos de uso: [Abrir](#)
2. Diagrama Entidad-Relación: [Abrir](#)
3. Diagrama de clases: [Abrir](#)
4. Diagrama de estados: [Abrir](#)

5. Diagrama de secuencia: [Abrir](#)

Anexo D — Estructura del repositorio

La estructura de carpetas principal del proyecto es la siguiente:

```

BrisasDeCalamuchita/
├── .github/workflows/ci.yml      # Pipeline de CI/CD
├── backend/
│   ├── src/                    # Código fuente del backend
│   │   ├── app.js              # Configuración de Express
│   │   ├── servidor.js         # Bootstrap del servidor
│   │   ├── config/             # env.js, bd.js
│   │   ├── rutas/              # Endpoints HTTP
│   │   ├── controladores/      # Lógica de los endpoints
│   │   ├── servicios/          # Lógica de negocio
│   │   ├── middlewares/        # Auth, errores, etc.
│   │   ├── tareas/             # Crons internos
│   │   └── validadores/        # Schemas Zod
│   ├── tests/                  # Tests unitarios y E2E
│   ├── migraciones/            # SQL de schema
│   ├── semillas/               # SQL de seeds
│   ├── scripts/                # Runner-sql, setup-env
│   ├── package.json
│   └── .env.ejemplo
├── frontend/
│   ├── src/
│   │   ├── App.jsx
│   │   ├── main.jsx
│   │   ├── paginas/            # Landing, Reservar, Admin, etc.
│   │   ├── componentes/        # Header, Footer, Calendario...
│   │   ├── api/                # Cliente HTTP y módulos
│   │   ├── contexto/           # ContextoAuth
│   │   ├── hooks/              # useApi
│   │   └── utiles/              # Helpers de fechas
│   ├── tests/                  # Tests con Vitest
│   ├── package.json
│   └── vite.config.js
├── docker-compose.yml
├── README.md
└── informe-final.docx

```

Anexo E — Resumen rápido de comandos

Acción	Comando
Levantar BD local	<code>docker compose up -d</code>
Detener BD local	<code>docker compose down</code>
Iniciar backend dev	<code>cd backend && npm run dev</code>
Iniciar frontend dev	<code>cd frontend && npm run dev</code>
Tests unitarios backend	<code>cd backend && npm run test:unit</code>
Tests E2E backend	<code>cd backend && npm run test:e2e</code>
Cobertura backend	<code>cd backend && npm run test:coverage</code>
Tests frontend	<code>cd frontend && npm test</code>
Build frontend producción	<code>cd frontend && npm run build</code>
Reset BD desarrollo	<code>cd backend && npm run resetear</code>

Anexo F — Notas finales para el equipo evaluador

Se sugiere consultar las secciones principales del informe en el siguiente orden recomendado, en función de los intereses específicos del equipo evaluador:

- Para evaluadores con foco en análisis y modelado: Bloques A y B (con los links a draw.io para exploración interactiva).
- Para evaluadores con foco en arquitectura e implementación: Bloque C, secciones "Arquitectura del sistema", "Stack tecnológico" y "Decisiones de diseño clave".
- Para evaluadores con foco en calidad y prácticas profesionales: Bloque C, secciones "Pruebas unitarias", "Pruebas de integración", "CI/CD y despliegue".
- Para evaluadores con foco en experiencia de usuario: Bloque C "Frontend en profundidad" y Bloque D "Manual de usuario".
- Para evaluadores que necesiten reproducir el entorno: Bloque D "Manual técnico — Instalación y puesta en marcha".

El autor queda a disposición durante el coloquio para profundizar en cualquier aspecto particular, demostrar el sistema en vivo, o discutir alternativas de diseño que fueron evaluadas pero no implementadas.

Anexo G — Uso de herramientas de inteligencia artificial

El desarrollo del presente proyecto incorporó el uso de herramientas de inteligencia artificial generativa, específicamente asistentes de lenguaje como ChatGPT y Claude, en instancias acotadas y bien delimitadas, con un rol de apoyo y no de sustitución del trabajo intelectual propio.

Etapas de ideación y definición del proyecto

Durante la fase inicial de exploración, antes de definir la naturaleza del sistema a desarrollar, se utilizó la IA como interlocutor para el brainstorming. Se le plantearon preguntas abiertas orientadas a identificar posibles problemáticas del entorno cotidiano que pudieran resolverse mediante una aplicación web. Un ejemplo representativo de este tipo de consulta fue el siguiente prompt:

“Estoy buscando ideas para desarrollar un sistema web como proyecto académico. Quiero que sea algo útil en la vida real, no un ejemplo genérico. ¿Qué problemas cotidianos podrían resolverse con una aplicación de gestión sencilla?”

La herramienta aportó un conjunto amplio de ideas como punto de partida. Entre ellas, la gestión digital de reservas para alojamientos turísticos resultó la más pertinente dado el contexto real disponible —la propiedad familiar Brisas de Calamuchita— y fue seleccionada por criterio propio del autor.

Definición del alcance funcional

Una vez elegida la temática, se consultó a la IA para explorar qué funcionalidades podría incluir un sistema de este tipo, con el objetivo de encontrar un equilibrio entre viabilidad académica y desafío técnico real. Un ejemplo de prompt utilizado en esta instancia fue:

“Estoy desarrollando un sistema de reservas para una casa de alquiler turístico. Es un proyecto académico de nivel técnico superior. ¿Qué funcionalidades podrían incluirse para que sea completo pero realizable en un semestre?”

Las respuestas de la herramienta operaron como catálogo de posibilidades. Se evaluaron, se descartaron las que excedían el alcance o no aportaban valor al caso de uso concreto, y se

adoptaron aquellas que resultaron pertinentes, como el modelo de estados de la reserva, el bloqueo temporal de fechas o el calendario visual de disponibilidad. En todos los casos, la decisión final de incluir o excluir cada funcionalidad fue tomada por el autor, con base en el análisis del problema real.

Mejoras en la interfaz de usuario

Durante el proceso de diseño visual, se consultó a la IA para obtener sugerencias sobre decisiones de UI que pudieran mejorar la experiencia del usuario final sin complejizar innecesariamente la implementación. Un ejemplo de prompt utilizado en esta instancia fue:

“Tengo un calendario de disponibilidad que usa colores para indicar el estado de cada fecha. ¿Qué convenciones de color son más intuitivas para un usuario que nunca usó el sistema? ¿Cómo puedo hacer que la leyenda sea clara sin ocupar demasiado espacio en pantalla?”

Las sugerencias recibidas sirvieron como referencia para evaluar alternativas de presentación. La codificación cromática final —gris para fechas pasadas, blanco para disponibles, amarillo para reservas pendientes y rojo para confirmadas— fue definida por el autor considerando tanto las recomendaciones obtenidas como la coherencia con la identidad visual general del sistema.

Orientación para las pruebas automatizadas

En la etapa de testing, se utilizó la IA para comprender qué aspectos del sistema convenía priorizar al momento de escribir las pruebas, dado que no era posible ni necesario cubrir la totalidad del código. Un ejemplo de consulta fue el siguiente:

“Estoy escribiendo pruebas unitarias y de integración para un sistema de reservas con Node.js y Jest. ¿Qué casos de prueba son los más representativos para validar la lógica de negocio de un sistema de este tipo?”

Las respuestas orientaron la selección de los casos de prueba más relevantes, pero la escritura del código de cada prueba, la definición de los datos de entrada y la interpretación de los resultados fueron realizadas íntegramente por el autor.

Formateo y revisión de la documentación

En la etapa de redacción final, se recurrió puntualmente a la IA para tareas de bajo impacto creativo: corrección de estilo y ortografía en párrafos específicos, sugerencias de estructura para secciones del informe, y verificación de coherencia en el formato de referencias bibliográficas según normas APA. Un ejemplo de consulta en esta etapa fue:

“Revisá este párrafo y corregí errores de redacción o estilo sin cambiar el contenido ni el tono formal del texto: [párrafo]”

El contenido, los argumentos y las decisiones documentadas en el presente informe son en su totalidad producto del trabajo del autor.

Criterio general de uso

En ningún caso se delegó en la IA la escritura completa de secciones del informe, la generación de código del sistema, ni el diseño de la arquitectura o el modelo de datos. Su rol se limitó a la generación de ideas como disparador inicial y a tareas auxiliares de revisión textual. Esta delimitación responde tanto a una decisión metodológica —aprender requiere resolver los problemas por cuenta propia— como al criterio establecido por la cátedra respecto del uso responsable de estas herramientas.

Anexo H — Documentos de trabajo de etapas anteriores

El presente anexo reúne los documentos de trabajo producidos en las etapas 1, 2 y 3 del proyecto integrador, a efectos de ofrecer al equipo evaluador una visión completa de la evolución del proyecto a lo largo del cuatrimestre.

Etapas 1 — Simulación de empresa y definición del software

Corresponde a la primera instancia evaluativa del proyecto. En ella se presentó la simulación de la empresa “Brisas de Calamuchita” bajo el nombre de equipo SierraByte, incluyendo la definición de misión, visión, valores, objetivos, contexto y problemática detectada, perfil del integrante y justificación del sistema a desarrollar.

Etapas 2 — Documento base, modelado y prototipo

Corresponde a la segunda instancia evaluativa. Contén el documento base del sistema con resumen ejecutivo, propósito, alcance, objetivos, usuarios y funcionalidades; el modelado UML completo (casos de uso, entidad–relación, clases, estados y secuencia); el análisis de soluciones similares; la evaluación de viabilidad; y el prototipo navegable de la interfaz desarrollado en Figma.

Los archivos originales correspondientes a ambas etapas se adjuntan junto al presente informe en el paquete de entrega del proyecto.

Etapas 3 — Video de presentación

El presente anexo incorpora el documento de trabajo correspondiente a la etapa 3 del proyecto integrador, cuya entrega principal consistió en la producción de un video de presentación del sistema. Dicho documento contiene el registro del proceso de preparación y la evidencia de la presentación realizada, en cumplimiento con los requerimientos de esa instancia de evaluación.

El archivo original correspondiente a esta etapa se adjunta junto al presente informe en el paquete de entrega del proyecto.

Agradecimientos

El autor desea expresar su sincero agradecimiento a los docentes de la cátedra de Práctica Profesional IV, Kevin Del Bello y Emir García Ontiveros, por el acompañamiento sostenido a lo largo de todo el semestre. Su disposición, sus devoluciones y la claridad con la que orientaron cada etapa del proyecto fueron determinantes para que este trabajo pudiera desarrollarse con solidez y llegar a una entrega completa.

Asimismo, se extiende el agradecimiento a todos los docentes de la Tecnicatura en Desarrollo de Software del Instituto de Formación Técnica Superior N.º 29, cuya labor a lo largo de la carrera sentó las bases técnicas, conceptuales y profesionales que hicieron posible la realización de este proyecto. Cada materia cursada aportó herramientas concretas que se vieron reflejadas, de una u otra forma, en el resultado final de este trabajo integrador.

Para el autor, el desarrollo de este proyecto representó una experiencia sumamente enriquecedora: gracias a este proyecto, fue posible integrar de manera coherente y aplicada los contenidos adquiridos a lo largo de toda la carrera, dando forma a una solución concreta, funcional y documentada que refleja el recorrido formativo realizado.

¡Muchas gracias a todo el equipo!

Jorge